

MANUAL COMPLETO DE INTRODUCCIÓN A LOS COMPUTADORES

Grado en Ingeniería Informática - 1º Curso

Este documento unifica todos los temas del plan de estudio de sistemas de numeración, diseño lógico combinacional/secuencial y arquitectura de procesadores en un único manual para facilitar su lectura, impresión o conversión a formatos como PDF.

Índice General

- **Bloque 1: Representación de la Información y Lógica Teórica**
 - [Tema 1: Representación de la Información y Conversión de Bases](#)
 - [Tema 2: Codificación de Números Enteros](#)
 - [Tema 3: Codificación de Números Reales y Estándar IEEE 754](#)
 - [Tema 4: Álgebra de Boole y Simplificación de Funciones](#)
- **Bloque 2: Electrónica Digital / Diseño de Circuitos**
 - [Tema 5: Sistemas Combinacionales I: Bloques Lógicos MSI](#)
 - [Tema 6: Sistemas Combinacionales II: Circuitos Aritméticos y la ALU](#)
 - [Tema 7: Elementos de Memoria Básicos: Biestables](#)
 - [Tema 8: Sistemas Secuenciales: Análisis y Diseño de Máquinas de Estados](#)
 - [Tema 9: Aplicaciones Secuenciales: Registros y Contadores](#)
- **Bloque 3: Arquitectura del Computador y Operación**
 - [Tema 10: Estructura Interna del Computador: El Modelo de Von Neumann](#)
 - [Tema 11: Organización del Procesador: Ruta de Datos y Unidad de Control](#)
 - [Tema 12: El Ciclo de Instrucción Paso a Paso](#)
 - [Tema 13: Introducción a la Programación en Lenguaje Ensamblador](#)
- **Secciones Finales**
 - [Glosario de Términos](#)
 - [Bibliografía Recomendada](#)

Tema 1: Representación de la Información y Conversión de Bases

La información que maneja un computador (textos, imágenes, sonido, programas) debe codificarse físicamente en un soporte binario. Para comprender cómo procesa y almacena datos el hardware, debemos estudiar primero los **sistemas de numeración posicionales** y los algoritmos matemáticos que nos permiten traducir cantidades entre distintas bases numéricas.

1. Sistemas de Numeración Posicionales

En un sistema posicional, un número se representa por una cadena de dígitos donde el valor de cada dígito depende de su posición respecto a la coma fraccionaria. Cada posición tiene asociado un peso que es una potencia de la **base (o radix) r** del sistema.

Un número genérico N en base r se expresa polinómicamente como:

$$N_r = \sum_{i=-m}^{n-1} d_i \cdot r^i = d_{n-1}r^{n-1} + \dots + d_0r^0 + d_{-1}r^{-1} + \dots + d_{-m}r^{-m}$$

donde:

- r : Base del sistema (número de dígitos disponibles).
- d_i : Dígitos permitidos ($0 \leq d_i < r$).
- n : Número de dígitos de la parte entera.
- m : Número de dígitos de la parte fraccionaria.

Bases Principales en Computación

Sistema	Base (r)	Dígitos permitidos
Decimal	10	$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
Binario	2	$\{0, 1\}$
Octal	8	$\{0, 1, 2, 3, 4, 5, 6, 7\}$
Hexadecimal	16	$\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A = 10, B = 11, C = 12, D = 13, E = 14, F = 15\}$

2. Algoritmos de Conversión entre Bases

2.1 Conversión a Base 10

Para pasar un número en cualquier base r a base 10, basta con desarrollar su polinomio característico sumando el producto de cada dígito por su peso decimal.

2.2 Conversión de Base 10 a Base r

El proceso se divide en dos partes:

- **Parte Entera (Divisiones Sucesivas):** Dividimos la parte entera entre la base r de forma reiterada. Los restos de cada división (leídos en orden inverso, de la última división a la primera) conforman el número entero en base r .
- **Parte Fraccionaria (Multiplicaciones Sucesivas):** Multiplicamos la parte fraccionaria por la base r . La parte entera del resultado será el dígito fraccionario correspondiente. Volvemos a tomar la parte fraccionaria resultante y multiplicamos por r , repitiendo hasta obtener parte fraccionaria cero o alcanzar la precisión deseada.

2.3 Conversiones Rápidas entre Bases Binarias ($r = 2^k$)

Dado que $8 = 2^3$ y $16 = 2^4$, podemos realizar conversiones instantáneas agrupando bits:

- **Binario a Octal:** Agrupamos los bits de 3 en 3 partiendo desde la coma hacia la izquierda y derecha. Cada grupo de 3 bits se sustituye directamente por su equivalente octal (0-7).
- **Binario a Hexadecimal:** Agrupamos los bits de 4 en 4. Cada grupo de 4 bits se sustituye directamente por su dígito hexadecimal equivalente (0-F).

3. El Toque Informático

¿Por qué los Computadores usan Binario y los Programadores Hexadecimal?

- **Binario en Hardware:** Es físicamente sencillo construir componentes electrónicos que distingan únicamente dos estados de tensión eléctrica estables: presencia de tensión (un "1"

lógico, ej. 5V o 1.2V) o ausencia de tensión (un "0" lógico, GND). Usar más niveles aumentaría drásticamente los errores por ruido térmico en las líneas de datos.

- **Hexadecimal en Software:** El binario es ilegible para humanos en secuencias largas. La memoria se organiza en **bytes (8 bits)**. Como $2^4 = 16$, un byte se puede representar exactamente con dos dígitos hexadecimales (por ejemplo, 11111111_2 es FF_{16}). Esto simplifica la visualización de direcciones de memoria (punteros) y códigos de colores en programación web (ej. `#FFFFFF` para blanco).

A continuación, implementamos en Python una utilidad que convierte un número decimal (con parte fraccionaria) a su representación binaria equivalente de forma analítica.

```
def decimal_a_binario(numero_dec, precision=10):
    parte_entera = int(numero_dec)
    parte_fraccionaria = numero_dec - parte_entera

    # 1. Parte entera por divisiones sucesivas
    bits_enteros = []
    temp = parte_entera
    if temp == 0:
        bits_enteros.append("0")
    while temp > 0:
        bits_enteros.append(str(temp % 2))
        temp = temp // 2
    bits_enteros.reverse() # Leemos de abajo a arriba

    # 2. Parte fraccionaria por multiplicaciones sucesivas
    bits_fracc = []
    temp_f = parte_fraccionaria
    while temp_f > 0 and len(bits_fracc) < precision:
        temp_f *= 2
        digito = int(temp_f)
        bits_fracc.append(str(digito))
        temp_f -= digito

    resultado_entero = "".join(bits_enteros)
    resultado_fracc = "".join(bits_fracc)

    if resultado_fracc:
        return f"{resultado_entero}.{resultado_fracc}"
    return resultado_entero

# Prueba
num = 45.625
print(f"Decimal: {num}")
print(f"Binario calculado: {decimal_a_binario(num)}")
```

4. Ejercicios Resueltos

Ejercicio 1

Convertir el número decimal 45.625_{10} a base binaria de forma analítica.

Solución:

1. Parte entera (45):

- $45/2 = 22 \Rightarrow$ Resto 1
- $22/2 = 11 \Rightarrow$ Resto 0
- $11/2 = 5 \Rightarrow$ Resto 1
- $5/2 = 2 \Rightarrow$ Resto 1
- $2/2 = 1 \Rightarrow$ Resto 0
- $1/2 = 0 \Rightarrow$ Resto 1

Leemos los restos de abajo a arriba: 101101_2 .

2. Parte fraccionaria (0.625):

- $0.625 \cdot 2 = 1.25 \Rightarrow$ Dígito 1 (nos queda 0.25)
- $0.25 \cdot 2 = 0.50 \Rightarrow$ Dígito 0 (nos queda 0.5)
- $0.50 \cdot 2 = 1.00 \Rightarrow$ Dígito 1 (nos queda 0.0, terminamos)

Leemos de arriba a abajo: $.101_2$.

3. Resultado: $45.625_{10} = 101101.101_2$.

Ejercicio 2

Convertir el número binario 1101011.011_2 a octal y hexadecimal mediante agrupamiento.

Solución:

1. A Octal (grupos de 3 bits):

- Parte entera (desde la coma a la izquierda): $\underbrace{001}_1 \underbrace{101}_5 \underbrace{011}_3$ (añadimos ceros a la izquierda para completar).
- Parte fraccionaria (desde la coma a la derecha): $\underbrace{011}_3$.
- Resultado: 153.3_8 .

2. A Hexadecimal (grupos de 4 bits):

- Parte entera: $\underbrace{0110}_6 \underbrace{1011}_{11=B}$ (añadimos ceros a la izquierda para completar).
 - Parte fraccionaria: $\underbrace{0110}_6$ (añadimos un cero a la derecha para completar).
 - Resultado: $6B.6_{16}$.
-

5. Ejercicios Propuestos

1. Convierte el número decimal 127.3125_{10} a base binaria y hexadecimal detallando los pasos de la conversión de la parte entera y la fraccionaria.
2. Halla el equivalente decimal de la cadena hexadecimal $A2C.8_{16}$ desarrollando su polinomio característico de potencias de 16.
3. Explica por qué algunas fracciones decimales exactas (como 0.1_{10} o 0.2_{10}) se convierten en fracciones periódicas infinitas en base binaria, y analiza qué problemas de precisión puede inducir esto en el software.

Tema 2: Codificación de Números Enteros

Para realizar cálculos aritméticos en un computador, no basta con codificar números naturales; debemos representar valores positivos y negativos. A lo largo de la historia de la informática se han diseñado varios métodos de codificación de enteros firmados (con signo). De todos ellos, el **Complemento a 2** se ha impuesto de forma universal en las Unidades Aritmético-Lógicas (ALUs) de los procesadores actuales debido a su eficiencia matemática para simplificar sumas y restas en un único circuito sumador básico.

1. Métodos de Codificación de Enteros Firmados (para N bits)

Consideramos una palabra binaria de longitud fija de N bits.

1.1 Signo-Magnitud (SM)

- El bit más significativo (MSB, a la izquierda) representa el signo: 0 para positivo (+) y 1 para negativo (-).
- Los restantes $N - 1$ bits representan el valor absoluto (magnitud) en binario puro.
- **Problemas:** Existe el doble cero ($+0 = 00000000_2$ y $-0 = 10000000_2$ para 8 bits), lo que complica las comprobaciones lógicas. La suma requiere circuitos separados para sumar y restar.
- **Rango:** $[-2^{N-1} + 1, 2^{N-1} - 1]$.

1.2 Complemento a 1 (C1)

- Los números positivos se representan igual que en binario puro.
- Los números negativos se obtienen **invirtiendo todos los bits** de su equivalente positivo (cambiando 0 por 1 y viceversa).
- **Problemas:** Sigue existiendo doble representación del cero ($+0$ y -0).
- **Rango:** $[-2^{N-1} + 1, 2^{N-1} - 1]$.

1.3 Complemento a 2 (C2)

- Los números positivos se representan igual que en binario puro.

- Los números negativos se obtienen tomando su equivalente positivo, **invirtiendo todos sus bits (C1) y sumando 1** al bit menos significativo (LSB, a la derecha).
- **Ventajas:**
 - **Cero único:** El cero se representa únicamente como 00000000_2 (para 8 bits). El acarreo sobrante al negar y sumar se descarta.
 - **Suma unificada:** La resta $A - B$ se ejecuta físicamente como la suma $A + (-B)$.
- **Rango:** $[-2^{N-1}, 2^{N-1} - 1]$ (permite representar un número negativo extra).

1.4 Exceso o Representación Sesgada (Excess- S)

Consiste en almacenar el número sumándole un valor constante llamado **sesgo (o exceso)** S de forma que todos los números almacenados sean positivos:

$$\text{Valor Codificado} = \text{Valor Real} + S$$

El sesgo habitual para N bits es $S = 2^{N-1}$ o $S = 2^{N-1} - 1$. Es de gran utilidad para codificar los exponentes en coma flotante.

2. Aritmética y Detección de Desbordamiento (Overflow) en C2

Al operar con un número fijo de N bits, el resultado de una suma de dos enteros firmados puede salirse del rango de representación máximo de la palabra binaria, lo que produce un **desbordamiento (overflow)** y devuelve un resultado erróneo.

Regla de Signos

- Si sumamos dos números de signos opuestos, **nunca** puede producirse desbordamiento.
- Si sumamos dos números del mismo signo (ambos positivos o ambos negativos), se produce desbordamiento si y solo si el resultado tiene el **signo opuesto** al de los sumandos.

Lógica de Compuertas en Hardware

En el interior de la ALU, el circuito detecta el desbordamiento de forma instantánea comparando los acarreos del bit de signo:

$$V = C_{in} \oplus C_{out}$$

donde C_{in} es el acarreo entrante al bit de signo (MSB) y C_{out} es el acarreo saliente del bit de signo. Si la operación XOR da 1, se activa la bandera de desbordamiento (Overflow Flag).

3. El Toque Informático

La Vulnerabilidad de Desbordamiento de Enteros en Software (Wrap-around)

En lenguajes de programación compila-directos como C o C++, el tipo `int` estándar suele codificarse en C2 usando 32 bits. El valor máximo representable es $2^{31} - 1 = 2.147.483.647$. Si sumamos 1 a este valor:

$$2.147.483.647 + 1 = -2.147.483.648$$

El valor de repente "salta" a su extremo negativo (`INT_MIN`) debido a que el bit de acarreo invade el bit de signo. Este comportamiento conocido como **wrap-around** es el origen de vulnerabilidades de seguridad críticas. Un atacante puede explotar un desbordamiento de enteros para saltarse comprobaciones de tamaño de búfer en memoria (`if (size1 + size2 < MAX_BUFFER)`), asignando un búfer minúsculo que luego provocará un desbordamiento de pila (stack overflow) al escribir.

A continuación, implementamos en Python una simulación del comportamiento de la aritmética en Complemento a 2 para 8 bits, incluyendo la detección de desbordamientos.

```
def int_to_c2(val, bits=8):
    # Rango en C2
    lim_inf = - (2 ** (bits - 1))
    lim_sup = (2 ** (bits - 1)) - 1

    if val < lim_inf or val > lim_sup:
        raise ValueError(f"Valor fuera de rango para {bits} bits.")

    if val >= 0:
        return f"{val:0{bits}b}"
    else:
        # Para negativos, hacemos el equivalente a módulo 2^bits
        return f"{(2**bits) + val:0{bits}b}"

# Simulación de suma en C2 de 8 bits: A + B
def sumar_c2(a, b, bits=8):
    lim_inf = - (2 ** (bits - 1))
    lim_sup = (2 ** (bits - 1)) - 1

    suma_real = a + b
    overflow = suma_real < lim_inf or suma_real > lim_sup

    # Simulación de truncamiento a 'bits' de longitud
    val_truncado = (a + b) % (2 ** bits)
    if val_truncado >= (2 ** (bits - 1)):
        val_final_firmado = val_truncado - (2 ** bits)
    else:
        val_final_firmado = val_truncado
```

```
print(f"Operación: {a} + {b}")
print(f"  Binario A: {int_to_c2(a, bits)}")
print(f"  Binario B: {int_to_c2(b, bits)}")
print(f"  Resultado obtenido: {int_to_c2(val_final_firmado, bits)} (Equivale a
{val_final_firmado} en base 10)")
print(f"  ¿Ocurrió desbordamiento (Overflow)? : {'Sí' if overflow else 'NO'}\n")

# Pruebas
sumar_c2(85, 20) # Sin desbordamiento
sumar_c2(85, 64) # Produce desbordamiento positivo (> 127)
```

4. Ejercicios Resueltos

Ejercicio 1

Representar el número entero -18_{10} utilizando 8 bits en los formatos de Signo-Magnitud, Complemento a 1 y Complemento a 2.

Solución:

1. **Representamos el número positivo $+18_{10}$ en binario puro (8 bits):**

- $18 = 16 + 2 = 00010010_2$.

2. **Signo-Magnitud (SM):**

- Cambiamos únicamente el bit más significativo (MSB) a 1 para indicar signo negativo:

$$SM = 10010010_2$$

3. **Complemento a 1 (C1):**

- Invertimos todos los bits de $+18$:

$$C1 = 11101101_2$$

4. **Complemento a 2 (C2):**

- Tomamos la representación de C1 y sumamos 1:

$$11101101_2 + 1 = 11101110_2$$

El resultado para -18_{10} en C2 de 8 bits es 11101110_2 .

Ejercicio 2

Realizar en C2 de 8 bits la suma de los enteros $85_{10} + 64_{10}$. Determinar el resultado binario obtenido y analizar si se produce desbordamiento.

Solución:

1. Codificar los sumandos a binario (C2):

- $85_{10} = 01010101_2$
- $64_{10} = 01000000_2$

2. Realizar la suma binaria:

```
  01010101   (85)
+ 01000000   (64)
-----
 10010101
```

3. Verificación del resultado y desbordamiento:

- Los sumandos son de signo positivo (el bit de signo de ambos es 0).
- El resultado binario obtenido es 10010101_2 . El bit de signo de este resultado es **1** (negativo).
- Como la suma de dos positivos dio como resultado un número negativo, **se ha producido desbordamiento (overflow)**.
- El valor decimal interpretado erróneamente en C2 de 8 bits para 10010101_2 es:

$$10010101_2 \Rightarrow (\text{Restamos } 1) = 10010100_2 \Rightarrow (\text{Invertimos}) = -01101011_2 = -107_{10}$$

El circuito de la ALU devuelve el valor erróneo -107_{10} en lugar de $+149_{10}$ debido a que $+149$ supera el límite superior de 8 bits en C2 ($+127$).

5. Ejercicios Propuestos

1. Codifica el número decimal -57_{10} en Complemento a 2 utilizando una longitud de palabra de 8 bits.
2. Dada la palabra binaria de 8 bits 10110011_2 , calcula su valor decimal equivalente asumiendo que está representada en: (a) Binario sin signo, (b) Signo-Magnitud, (c) Complemento a 2.
3. Realiza la operación de resta $34 - 72$ en C2 utilizando 8 bits (representándola como $34 + (-72)$) e indica el resultado binario obtenido comprobando si existe desbordamiento.

Tema 3: Codificación de Números Reales y Estándar IEEE 754

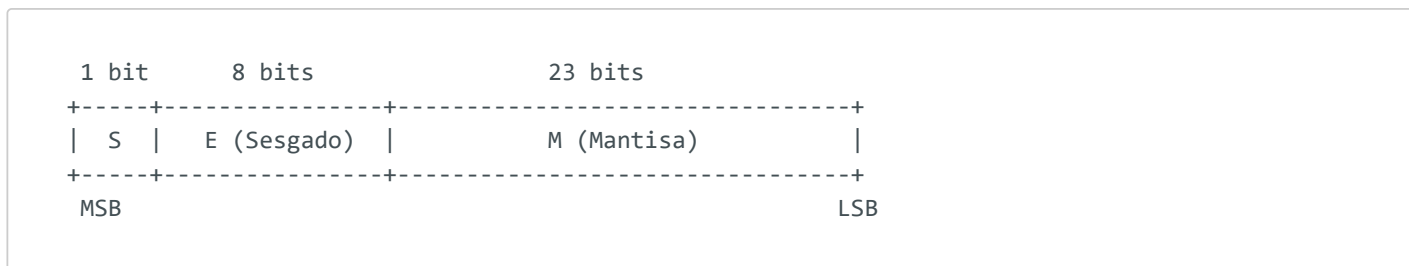
Representar números fraccionarios o extremadamente grandes en un computador exige un sistema que mueva la coma decimal de forma dinámica para optimizar la precisión. Mientras que la **coma fija** limita gravemente el rango de representación, el sistema de **coma flotante** (formalizado universalmente en el estándar **IEEE 754**) nos permite representar desde la distancia entre átomos hasta la distancia entre galaxias utilizando un número acotado de bits.

1. Coma Fija vs. Coma Flotante

- **Coma Fija:** Se reserva una cantidad fija de bits para la parte entera y otra para la parte fraccionaria. Por ejemplo, en una palabra de 8 bits con 4 bits enteros y 4 fraccionarios, el rango máximo es de $[0, 15.9375]$ con un paso constante de 0.0625. No sirve para representar valores muy grandes o muy pequeños.
- **Coma Flotante:** Utiliza notación científica binaria. Permite representar un número V mediante tres campos: un bit de signo, una mantisa (cifras significativas) y un exponente (que desplaza la coma a izquierda o derecha).

2. El Estándar IEEE 754 de Precisión Simple (32 bits)

En el formato de precisión simple de 32 bits, los bits se distribuyen en tres campos diferenciados:



1. **Signo (S):** 1 bit (bit 31). Define si el número es positivo ($S = 0$) o negativo ($S = 1$).
2. **Exponente (E):** 8 bits (bits 23 a 30). Se almacena en representación de **exceso 127** (sesgado). El exponente real e es:

$$e = E - 127$$

3. **Mantisa (M):** 23 bits (bits 0 a 22). Almacena la parte fraccionaria de una mantisa normalizada. El estándar asume un **1 implícito** (bit oculto) a la izquierda de la coma que no

se almacena en memoria para ahorrar espacio. La mantisa real es:

$$\text{Mantisa Real} = 1 + M = 1.f \quad (\text{donde } f \text{ es la fracción almacenada})$$

La fórmula general de interpretación para números normalizados es:

$$V = (-1)^S \cdot (1.M) \cdot 2^{E-127}$$

3. Rangos Especiales de Exponente en IEEE 754

El estándar reserva los valores extremos del exponente ($E = 0$ y $E = 255$) para codificar casos especiales:

Campo Exponente (E)	Campo Mantisa (M)	Interpretación / Tipo de Valor
$0 < E < 255$	Cualquier valor	Número Normalizado (aritmética estándar).
$E = 0$	$M = 0$	Cero (+0.0 o -0.0, según bit de signo).
$E = 0$	$M \neq 0$	Número Desnormalizado ($V = (-1)^S \cdot (0.M) \cdot 2^{-126}$). Permite una caída gradual de precisión cerca del cero.
$E = 255$	$M = 0$	Infinito ($+\infty$ o $-\infty$, según bit de signo). Resultados como divisiones entre cero ($x/0$).
$E = 255$	$M \neq 0$	NaN (Not a Number) . Resultados matemáticamente indeterminados (como $0/0$ o $\sqrt{-1}$).

4. El Toque Informático

Inestabilidad de Comparación de Reales y la Necesidad de Épsilon

En computación, la representación de coma flotante introduce errores de redondeo porque muchas fracciones decimales exactas (como 0.1_{10} o 0.2_{10}) se convierten en binario en expresiones periódicas infinitas. En memoria de 32 bits, se truncan de forma irreversible. Por este motivo:

```
// ERROR CRÍTICO DE PROGRAMACIÓN
if (x == 0.3) { ... }
```

La condición anterior casi siempre será evaluada como **falsa** debido a las pequeñas diferencias de redondeo del bit menos significativo.

- **Solución:** Las comparaciones de números reales en programación deben realizarse definiendo un margen de tolerancia llamado **Épsilon** (ϵ):

```
// SOLUCIÓN CORRECTA
if (abs(x - 0.3) < 1e-6) { ... }
```

A continuación, implementamos en Python un conversor que traduce un número real de precisión simple a su representación binaria IEEE 754 de 32 bits.

```
import struct

def float_to_ieee754(num):
    # struct.pack('f', num) empaqueta el número como float de 32 bits (C float)
    # y 'I' lo interpreta como un entero sin signo de 32 bits
    packed = struct.pack('>f', num)
    val_int = struct.unpack('>I', packed)[0]

    bin_str = f"{val_int:032b}"

    signo = bin_str[0]
    exponente = bin_str[1:9]
    mantisa = bin_str[9:]

    print(f"Número Real: {num}")
    print(f"  Bit de Signo (S): {signo}")
    print(f"  Exponente (E):      {exponente} (Decimal: {int(exponente, 2)})")
    print(f"  Mantisa (M):         {mantisa}")
    print(f"  Representación Hexadecimal: 0x{val_int:08X}")
    return bin_str

# Prueba
float_to_ieee754(-6.625)
```

5. Ejercicios Resueltos

Ejercicio 1

Representar el número decimal -6.625_{10} en el estándar IEEE 754 de precisión simple.

Solución:

1. **Determinar el bit de signo:** El número es negativo, por tanto: $S = 1$.
2. **Convertir el valor absoluto a binario:** $6_{10} = 110_2$ y $0.625_{10} = .101_2 \Rightarrow 6.625_{10} = 110.101_2$.
3. **Normalizar la mantisa:** Desplazamos la coma hacia la izquierda hasta que quede un único dígito diferente de cero a su izquierda:

$$110.101_2 = 1.10101_2 \cdot 2^2$$

- La mantisa normalizada es 1.10101.
- La parte fraccional a almacenar en los 23 bits de memoria es:
10101000000000000000000₂ (completando con ceros a la derecha).
- El exponente real es: $e = 2$.

4. **Calcular el exponente sesgado (E):**

$$E = e + 127 = 2 + 127 = 129_{10}$$

Convertimos 129 a binario de 8 bits: $129/2 \dots \Rightarrow 10000001_2$.

5. **Ensamblar la palabra de 32 bits:**

$$\text{Palabra} = \underbrace{1}_S \underbrace{10000001}_E \underbrace{101010000000000000000000}_M$$

En hexadecimal, agrupando de 4 en 4: $1100\ 0000\ 1101\ 0100 \dots_2 = C0D40000_{16}$.

Ejercicio 2

Decodificar la palabra hexadecimal de 32 bits $0x40B00000$ para hallar su valor decimal real según IEEE 754.

Solución:

1. **Convertir el hexadecimal a binario:** $4 = 0100$, $0 = 0000$, $B = 1011$, $0 = 0000 \dots$

$$\text{Binario} = 0100\ 0000\ 1011\ 0000 \dots 0000_2$$

2. **Extraer los campos:**

- $S = 0$ (número positivo).
- $E = 10000001_2 = 129_{10}$.
- $M = 01100000 \dots 0_2 = 0.011_2$ (en base decimal: $0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} = 0.25 + 0.125 = 0.375$).

3. **Calcular el exponente real (e):**

$$e = E - 127 = 129 - 127 = 2$$

4. **Calcular el valor real:**

$$V = (-1)^0 \cdot (1 + 0.375) \cdot 2^2 = 1.375 \cdot 4 = 5.5_{10}$$

La palabra codifica el número real $+5.5_{10}$.

6. Ejercicios Propuestos

1. Codifica el número real $+0.75_{10}$ en el estándar IEEE 754 de precisión simple y expresa su resultado en formato binario de 32 bits y hexadecimal.
2. Dada la palabra binaria `0xBF800000` en IEEE 754 de precisión simple, realiza la decodificación paso a paso para hallar el valor real en base 10.
3. Investiga el formato de precisión doble del estándar IEEE 754. Detalla cuántos bits reserva para cada campo (signo, exponente, mantisa), qué valor de sesgo utiliza y cuál es su ventaja respecto a la precisión simple.

Tema 4: Álgebra de Boole y Simplificación de Funciones

El álgebra de Boole es el marco matemático utilizado para formalizar el comportamiento de las variables lógicas binarias. Define las reglas matemáticas que gobiernan las compuertas físicas de los circuitos de los computadores. Puesto que cada compuerta lógica representa un coste físico en silicio (transistores), aprender a simplificar funciones lógicas mediante leyes algebraicas y **Mapas de Karnaugh** es crucial para optimizar el coste y rendimiento del hardware.

1. Postulados y Teoremas del Álgebra de Boole

El álgebra de Boole opera sobre un conjunto $\{0, 1\}$ con tres operaciones básicas:

- Suma Lógica (OR):** $A + B$ (se activa si A o B es 1).
- Producto Lógico (AND):** $A \cdot B$ o AB (se activa únicamente si ambos son 1).
- Inversión (NOT):** $\bar{A}$ o A' (invierte el valor de A).

Leyes Fundamentales

- Identidad:** $A + 0 = A$, $A \cdot 1 = A$
- Nulo (Dominancia):** $A + 1 = 1$, $A \cdot 0 = 0$
- Idempotencia:** $A + A = A$, $A \cdot A = A$
- Complementariedad:** $A + \bar{A} = 1$, $A \cdot \bar{A} = 0$
- Leyes de De Morgan:**

$$\overline{A + B} = \bar{A} \cdot \bar{B}$$

$$\overline{A \cdot B} = \bar{A} + \bar{B}$$

2. Formas Canónicas de una Función Lógica

Una función lógica relaciona variables de entrada binarias con una salida. Se puede representar formalmente de dos formas canónicas a partir de su tabla de verdad:

2.1 Primera Forma Canónica (Suma de Productos - Minterms)

Consiste en realizar una operación **OR** de los **Minterms** correspondientes a las filas donde la salida es **1**. Un minterm es un producto lógico (AND) donde aparecen todas las variables (afirmadas si valen 1, negadas si valen 0).

$$F(A, B, C) = \sum m(1, 3, 5)$$

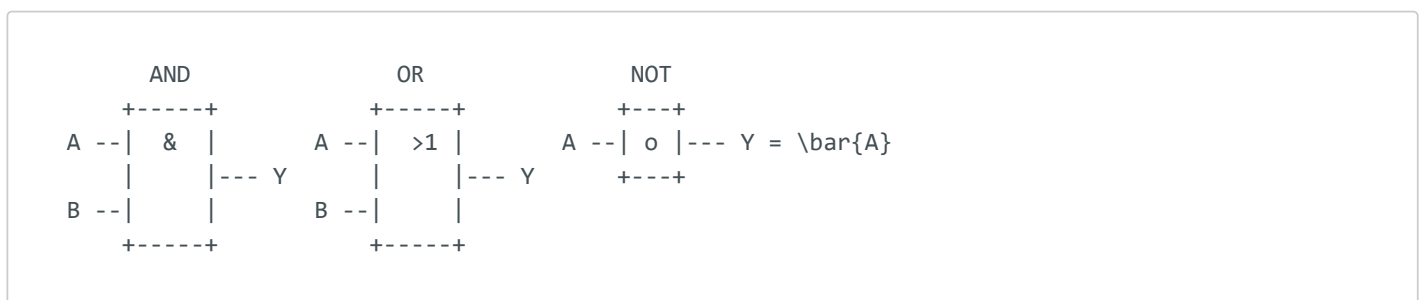
2.2 Segunda Forma Canónica (Producto de Sumas - Maxterms)

Consiste en realizar una operación **AND** de los **Maxterms** correspondientes a las filas donde la salida es **0**. Un maxterm es una suma lógica (OR) de todas las variables (negadas si valen 1, afirmadas si valen 0).

$$F(A, B, C) = \prod M(0, 2, 4, 6, 7)$$

3. Puertas Lógicas Fundamentales

Las puertas lógicas son los bloques electrónicos básicos que realizan físicamente las operaciones algebraicas en el silicio.



- **NAND y NOR (Puertas Universales):** Las puertas NAND y NOR son de extrema importancia práctica. Cualquier circuito digital, por complejo que sea, puede implementarse utilizando exclusivamente compuertas NAND o compuertas NOR. Físicamente, en tecnología CMOS, una compuerta NAND requiere menos transistores y es más rápida que una AND pura.

4. Simplificación de Funciones mediante Mapas de Karnaugh (K-Maps)

El mapa de Karnaugh es una representación visual matricial de la tabla de verdad estructurada de forma que las celdas adyacentes difieren exactamente en un único bit (usando **Código Gray**). Esta adyacencia geométrica permite aplicar visualmente la ley de simplificación:

$$XY + X\bar{Y} = X(Y + \bar{Y}) = X$$

Reglas de Agrupamiento

1. Los grupos de celdas con valor "1" deben ser de tamaño potencia de dos: 1, 2, 4, 8 o 16 celdas.
2. Debemos hacer los grupos **lo más grandes posibles** y el **menor número posible** de ellos.
3. El mapa se "enrolla" de forma que los bordes izquierdo y derecho son adyacentes, así como el superior e inferior.
4. **Términos Indiferentes (Don't Care, "X")**: Representan estados de entrada imposibles o salidas que no importan. Se pueden tratar como 1 o 0 de forma conveniente para hacer grupos más grandes.

5. El Toque Informático

Optimización de Área y Consumo de Silicio en Microchips

Cada compuerta lógica implementada en silicio tiene asociado un tamaño de área y consume energía disipada en calor.

- Una función lógica compleja sin simplificar puede requerir 10 compuertas lógicas (unos 40 transistores).
- Tras simplificarla mediante mapas de Karnaugh, la misma función exacta puede requerir solo 2 compuertas (8 transistores).

Simplificar circuitos ahorra área en la oblea de silicio (permitiendo fabricar más microchips por lote), reduce el consumo de energía (alargando la batería de los dispositivos móviles) y disminuye el **retardo de propagación** (permitiendo que el procesador funcione a una frecuencia de reloj superior).

A continuación, implementamos en Python una simulación que genera la tabla de verdad y los minterms de una función de votación mayoritaria de tres variables.

```
# Función lógica de votación mayoritaria: 1 si al menos dos entradas son 1
def votacion_mayoritaria(a, b, c):
    return (a and b) or (a and c) or (b and c)
```

```
# Impresión de la Tabla de Verdad e identificación de Minterms
print("| A | B | C | Salida (Y) | Minterm Asociado |")
print("|---|---|---|-----|-----|")

minterms = []
for a in [0, 1]:
    for b in [0, 1]:
        for c in [0, 1]:
            y = int(votacion_mayoritaria(a, b, c))
            min_str = ""
            if y == 1:
                # Construimos el formato del minterm
                char_a = 'A' if a else 'A\''
                char_b = 'B' if b else 'B\''
                char_c = 'C' if c else 'C\''
                min_str = f"{char_a}{char_b}{char_c}"
                min_idx = a*4 + b*2 + c
                minterms.append(f"m{min_idx}")
            print(f"| {a} | {b} | {c} | {y} | {min_str:8s} |")

print(f"\nPrimera Forma Canónica (Suma de Productos): Y = " + " + ".join(minterms))
```

6. Ejercicios Resueltos

Ejercicio 1

Dada una función lógica de tres variables $F(A, B, C)$ definida por los minterms $F = \sum m(1, 3, 5, 7)$, escribir su primera forma canónica y simplificar la expresión mediante álgebra booleana.

Solución:

- Primera forma canónica (Minterms):** Escribimos la suma de productos correspondiente a los índices indicados:

- $m_1(001) \Rightarrow \bar{A}\bar{B}C$
- $m_3(011) \Rightarrow \bar{A}BC$
- $m_5(101) \Rightarrow A\bar{B}C$
- $m_7(111) \Rightarrow ABC$

$$F = \bar{A}\bar{B}C + \bar{A}BC + A\bar{B}C + ABC$$

- Simplificación Algebraica:**

- Agrupamos de dos en dos y sacamos factor común:

$$F = \bar{A}C(\bar{B} + B) + AC(\bar{B} + B)$$

- Como $B + \bar{B} = 1$:

$$F = \bar{A}C(1) + AC(1) = \bar{A}C + AC$$

- Volvemos a sacar factor común C :

$$F = C(\bar{A} + A)$$

- Como $A + \bar{A} = 1$:

$$F = C(1) = C$$

La función simplificada equivale a la variable de entrada C .

Ejercicio 2

Simplificar mediante un Mapa de Karnaugh la función de 3 variables: $F(A, B, C) = \sum m(3, 4, 5, 7)$.

Solución:

1. **Construir el Mapa de Karnaugh (Gray Code):** Filas representan A (0, 1). Columnas representan BC (00, 01, 11, 10).

	BC				
	00	01	11	10	
	+---+---+---+---+				
A 0	0	0	1	0	(Fila A=0: m0, m1, m3, m2)
	+---+---+---+---+				
1	1	1	1	0	(Fila A=1: m4, m5, m7, m6)
	+---+---+---+---+				

2. **Agrupar los 1s:**

- **Grupo 1 (Tamaño 2):** Celda ($A = 0, BC = 11$) y celda ($A = 1, BC = 11$). Corresponde a los minterms $m3$ y $m7$. En este grupo, la variable A cambia de 0 a 1 (se elimina), mientras que B y C permanecen constantes en 1. El término obtenido es BC .
- **Grupo 2 (Tamaño 2):** Celdas de la fila $A = 1$, columnas $BC = 00$ y $BC = 01$. Corresponde a los minterms $m4$ y $m5$. En este grupo, A es constante en 1. B permanece constante en 0 y C cambia de 0 a 1 (se elimina). El término obtenido es $A\bar{B}$.

3. **Resultado:** Sumamos los términos de los grupos óptimos:

$$F_{\text{simplificada}} = BC + A\bar{B}$$

7. Ejercicios Propuestos

1. Dibuja la tabla de verdad y obtén la primera forma canónica (Minterms) de una puerta lógica XOR de dos entradas.

2. Simplifica mediante un Mapa de Karnaugh la siguiente función lógica de 4 variables:

$$F(A, B, C, D) = \sum m(0, 2, 8, 10, 5, 7, 13, 15)$$

3. Demuestra mediante compuertas NAND elementales cómo construir un circuito equivalente a: (a) un inversor NOT, (b) una puerta AND de dos entradas.

Tema 5: Sistemas Combinacionales I: Bloques Lógicos MSI

Un sistema digital es **combinacional** si sus salidas en cualquier instante de tiempo dependen exclusivamente de los valores de sus entradas en ese mismo instante. Para facilitar el diseño de sistemas complejos, la industria agrupa compuertas básicas en bloques funcionales estándar de Mediana Escala de Integración (MSI - Medium Scale Integration). Estos bloques (codificadores, decodificadores, multiplexores) son los componentes básicos del enrutamiento de datos en el procesador.

1. Codificadores y Decodificadores

1.1 Codificadores

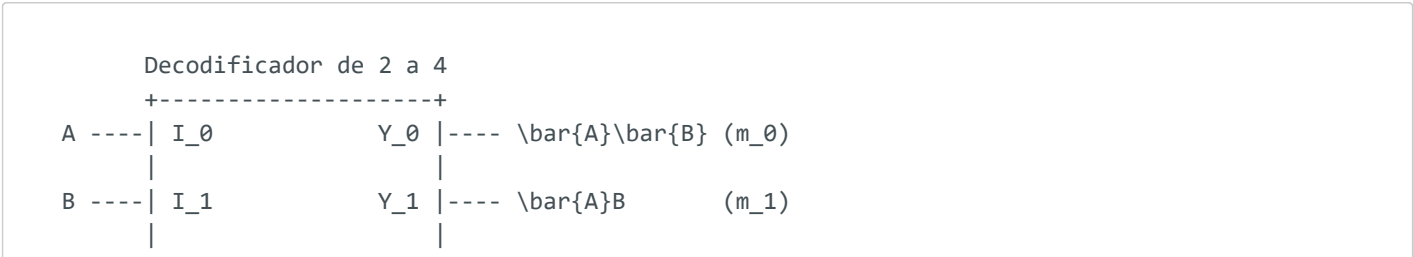
Dispositivo que realiza la operación inversa a un decodificador. Tiene 2^n líneas de entrada y n líneas de salida. Su función es representar en binario cuál de las entradas está activa.

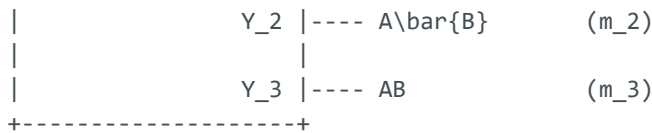
- **Codificadores con Prioridad:** En un codificador básico, si dos entradas se activan a la vez, el resultado es erróneo. Los codificadores con prioridad resuelven esto asignando prioridad a la entrada con el índice más alto. Además, incluyen una salida de control $\overline{V}$ (Valid) que indica si hay al menos una entrada activa.

1.2 Decodificadores

Dispositivo con n entradas y 2^n salidas. Su función consiste en activar únicamente la salida correspondiente al código binario introducido en las entradas.

- **Decodificador de 3 a 8:** Si introducimos el código 011_2 (3 decimal) en las entradas, se activa únicamente la salida Y_3 , quedando las demás en 0.
- **Implementación de funciones lógicas:** Dado que cada salida de un decodificador representa un **minterm** de las variables de entrada, podemos implementar cualquier función lógica sumando (mediante una compuerta OR) las salidas correspondientes a los minterms de la función.

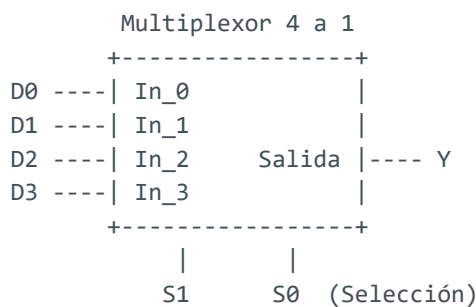




2. Multiplexores (MUX) y Demultiplexores (DEMUX)

2.1 Multiplexores

Un multiplexor es un **selector de datos**. Tiene 2^n entradas de datos, n entradas de selección (que actúan como control) y una única salida. El código binario en las entradas de selección determina cuál de las entradas de datos se conecta físicamente a la salida.



- **Implementación de funciones lógicas:** Un multiplexor de 2^n entradas de datos puede implementar cualquier función lógica de n variables conectando las variables directamente a las entradas de selección y fijando a "1" o "0" las entradas de datos según convenga.

2.2 Demultiplexores

Realiza la operación inversa. Tiene una única entrada de datos, n entradas de selección y 2^n salidas. Dirige el dato de la entrada única hacia la salida seleccionada por el bus de control.

3. El Toque Informático

Decodificadores de Memoria y Señal Chip Select (CS)

En los computadores, la memoria principal está compuesta por múltiples chips de memoria integrados físicamente en la placa base. Cuando el procesador quiere leer o escribir un dato, envía una dirección por el bus de direcciones (Address Bus).

- Las líneas de bits inferiores de la dirección se conectan a todos los chips para seleccionar la celda de memoria exacta (fila/columna).
- Las líneas de bits superiores (que identifican el rango de memoria) se conectan a un **decodificador binario**.
- La salida correspondiente del decodificador activa la patilla **Chip Select (CS)** de un único chip específico, apagando los demás para evitar colisiones de bus en las líneas de datos.

A continuación, implementamos en Python una simulación lógica de un Codificador con Prioridad de 4 a 2 con salida de validación.

```
def codificador_prioridad_4_a_2(entradas):
    # entradas es una lista de 4 booleanos: [I3, I2, I1, I0]
    # Retorna (Y1, Y0, V)

    if entradas[0]:      # Mayor prioridad para I3
        return 1, 1, 1
    elif entradas[1]:    # I2
        return 1, 0, 1
    elif entradas[2]:    # I1
        return 0, 1, 1
    elif entradas[3]:    # I0
        return 0, 0, 1
    else:                # Ninguna activa
        return 0, 0, 0

# Pruebas de simulación
entradas_test = [
    [0, 0, 0, 0], # Ninguna activa
    [0, 0, 1, 0], # Solo I1 activa
    [1, 1, 0, 0]  # I3 e I2 activas a la vez (debe ganar I3 por prioridad)
]

for ent in entradas_test:
    y1, y0, v = codificador_prioridad_4_a_2(ent)
    print(f"Entradas [I3, I2, I1, I0]: {ent} -> Salidas Y1Y0: {y1}{y0}, V (Válido): {v}")
```

4. Ejercicios Resueltos

Ejercicio 1

Implementar la función lógica $F(A, B, C) = \sum m(1, 2, 4, 7)$ utilizando un decodificador de 3 a 8 y una puerta OR externa.

Solución:

1. **Analizar el decodificador:** El decodificador de 3 a 8 tiene tres entradas (A, B, C) y 8 salidas (Y_0 a Y_7). Cada salida Y_i se activa únicamente cuando se introduce el minterm correspondiente a su índice en binario.
2. **Identificar salidas a conectar:** La función se activa para los minterms 1, 2, 4 y 7. Por tanto, conectamos las salidas Y_1, Y_2, Y_4 e Y_7 del decodificador a las entradas de una compuerta OR de 4 entradas.
3. **Esquema circuital (conceptual):**
 - Entradas $A, B, C \rightarrow$ Conectadas a los pines de selección del decodificador.
 - $F = Y_1 + Y_2 + Y_4 + Y_7$.

Cuando el código de entrada coincide con uno de los minterms indicados, la correspondiente salida del decodificador pasa a 1, activando la salida final de la puerta OR a 1.

Ejercicio 2

Implementar la misma función lógica $F(A, B, C) = \sum m(1, 2, 4, 7)$ utilizando un multiplexor de 8 a 1.

Solución:

1. **Analizar el multiplexor de 8 a 1:** Tiene 8 entradas de datos (D_0 a D_7), 3 entradas de selección (S_2, S_1, S_0) y 1 salida (Y).
2. **Conectar las variables:** Conectamos las tres variables del problema (A, B, C) a las líneas de selección:
 - $S_2 = A, S_1 = B, S_0 = C$.
3. **Configurar las entradas de datos:** Fijamos los valores de las entradas de datos de forma estática según la tabla de verdad de la función:
 - Para los minterms presentes (1, 2, 4, 7), conectamos las correspondientes entradas a "1" lógico (V_{cc}):

$$D_1 = D_2 = D_4 = D_7 = 1$$

- Para los minterms ausentes (0, 3, 5, 6), conectamos las correspondientes entradas a "0" lógico (GND):

$$D_0 = D_3 = D_5 = D_6 = 0$$

5. Ejercicios Propuestos

1. Dibuja el esquema lógico de un decodificador de 2 a 4 utilizando compuertas básicas AND y NOT.

2. Explica conceptualmente cómo se puede implementar una función lógica de 4 variables utilizando únicamente un multiplexor de 8 a 1 (pista: conectando una de las variables a las entradas de datos en lugar de fijarlas a valores constantes, método del árbol de selección).
3. ¿Cuál es la función de las entradas de habilitación (Enable) en los decodificadores MSI comerciales y cómo se usan para conectar múltiples chips en cascada?

Tema 6: Sistemas Combinacionales II: Circuitos Aritméticos y la ALU

La capacidad de un computador para ejecutar cálculos numéricos y lógicos reside en la **Unidad Aritmético-Lógica (ALU)**. La ALU es un sistema puramente combinacional compuesto por bloques aritméticos (sumadores, restadores, comparadores) y lógicos (puertas AND, OR, XOR) cuya salida es seleccionada dinámicamente mediante una señal de control. Estudiar los circuitos sumadores es la puerta de entrada para comprender cómo procesa físicamente la información el silicio.

1. Circuitos Sumadores Básicos

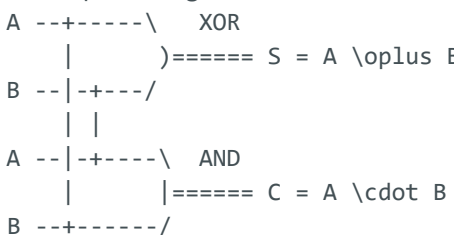
1.1 Semisumador (Half-Adder)

Suma dos bits de entrada (A y B) y produce dos salidas: el bit de suma (S) y el acarreo de salida (C).

Tabla de Verdad

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Esquema Lógico



- Ecuaciones:

$$S = A \oplus B$$

$$C = A \cdot B$$

1.2 Sumador Completo (Full-Adder)

Suma tres bits: los dos operandos (A y B) y el acarreo proveniente de la etapa de suma anterior (C_{in}). Produce el bit de suma (S) y el acarreo de salida (C_{out}).

- Ecuaciones:

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = A \cdot B + (A \oplus B) \cdot C_{in}$$

2. Sumadores de Múltiples Bits (Paralelos)

Para sumar palabras de N bits se conectan varios sumadores completos en cascada:

2.1 Sumador con Propagación de Acarreo (Ripple Carry Adder - RCA)

El acarreo de salida de cada etapa se conecta directamente al acarreo de entrada de la etapa siguiente.

- **Problema:** Retardo proporcional a la longitud de palabra ($O(N)$). Las últimas etapas deben esperar a que se calcule el acarreo de todas las anteriores, limitando la velocidad máxima del procesador.

2.2 Sumador con Anticipación de Acarreo (Carry Lookahead Adder - CLA)

Calcula los acarreos de todas las etapas en paralelo en tiempo constante $O(1)$ mediante dos funciones lógicas previas:

- Generador de Acarreo: $G_i = A_i \cdot B_i$ (se genera acarreo si ambos son 1).
 - Propagador de Acarreo: $P_i = A_i \oplus B_i$ (se propaga el acarreo de entrada).
-

3. Estructura Interna de una ALU Elemental

Una ALU básica consta de:

1. **Ruta de Datos:** Un sumador/restador paralelo y compuertas lógicas en paralelo.
 2. **Selector (Multiplexor):** Toma las salidas de todos los bloques anteriores y, mediante un bus de selección de instrucción (código de operación u OPCODE), redirige una de ellas hacia el bus de salida final.
-

4. El Toque Informático

El Registro de Estado (FLAGS Register)

Las ALUs no solo devuelven el resultado del cálculo, sino que activan señales de estado de 1 bit llamadas **Flags** (banderas) que se guardan en el Registro de Estado de la CPU tras cada operación:

- **Zero Flag (*Z*):** Se pone a 1 si el resultado de la operación es exactamente cero.
- **Sign Flag (*S* o *N*):** Copia el bit más significativo (MSB) del resultado, indicando si es negativo.
- **Carry Flag (*C*):** Se activa si la suma produce un acarreo saliente del MSB (operaciones sin signo).
- **Overflow Flag (*V* o *O*):** Se activa si ocurre un desbordamiento aritmético en Complemento a 2.

Estas banderas son el corazón de la bifurcación lógica: cuando programas un `if (a == b)` en alto nivel, el compilador realiza una resta en la ALU (`a - b`). Si la resta es cero, se activa el flag *Z*. La CPU lee este flag y ejecuta un salto condicional de instrucción (`JZ` - Jump if Zero).

A continuación, implementamos en Python una simulación lógica de un Sumador por Propagación de Acarreo (RCA) de 4 bits.

```
def full_adder(a, b, c_in):
    # Sumador completo de 1 bit
    s = a ^ b ^ c_in
    c_out = (a & b) | ((a ^ b) & c_in)
    return s, c_out

def ripple_carry_adder_4bits(bin_a, bin_b):
    # bin_a y bin_b son listas de 4 bits, ej. [0, 1, 0, 1] (MSB a la izquierda)
    # Volteamos para procesar desde el LSB (derecha) al MSB (izquierda)
    a = list(reversed(bin_a))
    b = list(reversed(bin_b))

    suma = []
    c = 0 # Acarreo inicial

    for i in range(4):
        s, c = full_adder(a[i], b[i], c)
        suma.append(s)

    suma.reverse() # Volvemos a colocar en orden MSB -> LSB
    return suma, c

# Suma: 5 (0101) + 6 (0110) = 11 (1011)
op1 = [0, 1, 0, 1]
op2 = [0, 1, 1, 0]
res, c_out = ripple_carry_adder_4bits(op1, op2)

print(f"Suma: {op1} + {op2}")
print(f" Resultado de la suma (S): {res}")
print(f" Acarreo de salida final (C_out): {c_out}")
```

5. Ejercicios Resueltos

Ejercicio 1

Deducir las ecuaciones del sumador completo (Full-Adder) a partir de su tabla de verdad por Minterms.

Solución:

1. Dibujamos la tabla de verdad:

A	B	C_{in}	S	C_{out}	Minterm
0	0	0	0	0	-
0	0	1	1	0	$m_1 = \bar{A}\bar{B}C_{in}$
0	1	0	1	0	$m_2 = \bar{A}B\bar{C}_{in}$
0	1	1	0	1	$m_3 = \bar{A}BC_{in}$
1	0	0	1	0	$m_4 = A\bar{B}\bar{C}_{in}$
1	0	1	0	1	$m_5 = A\bar{B}C_{in}$
1	1	0	0	1	$m_6 = AB\bar{C}_{in}$
1	1	1	1	1	$m_7 = ABC_{in}$

2. Ecuación para S (Suma):

$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

Simplificamos:

$$S = \bar{A}(\bar{B}C_{in} + B\bar{C}_{in}) + A(\bar{B}\bar{C}_{in} + BC_{in})$$

Identificamos el XOR ($\oplus$) y XNOR ($\odot$):

$$S = \bar{A}(B \oplus C_{in}) + A(\overline{B \oplus C_{in}}) = A \oplus B \oplus C_{in}$$

3. Ecuación para C_{out} (Acarreo):

$$C_{out} = \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in}$$

Simplificamos:

$$C_{out} = C_{in}(\bar{A}B + A\bar{B}) + AB(\bar{C}_{in} + C_{in}) = (A \oplus B)C_{in} + AB$$

Quedan demostradas las ecuaciones de diseño del sumador completo.

Ejercicio 2

Esbozar el bloque lógico de control de una ALU de 1 bit con dos líneas de selección (S_1, S_0) para realizar las operaciones: $00 \rightarrow \text{AND}$, $01 \rightarrow \text{OR}$, $10 \rightarrow \text{Suma}$, $11 \rightarrow \text{Resta}$.

Solución: El circuito consta de los siguientes bloques conectados en paralelo:

1. Compuerta AND (entrada A, B).
 2. Compuerta OR (entrada A, B).
 3. Sumador completo (entradas A, B, C_{in}).
 4. Para la resta ($A - B$), invertimos la entrada B mediante una puerta NOT si la señal es de resta ($S_1 S_0 = 11$). Físicamente se implementa alimentando B a una compuerta XOR controlada por S_0 (si $S_0 = 1$, B se invierte a $\bar{B}$).
 5. Las salidas de la compuerta AND, OR, y el bit de suma se conectan a un multiplexor de 4 a 1.
 6. Las líneas de selección del multiplexor se conectan a S_1 y S_0 . La salida seleccionada del multiplexor será la salida de la ALU.
-

6. Ejercicios Propuestos

1. Dibuja el diagrama de bloques de un sumador Ripple Carry de 4 bits interconectando 4 bloques de sumadores completos elementales.
2. Explica cómo se puede construir un circuito restador completo de 1 bit a partir de la modificación de los acarreos del sumador completo (restador con préstamo o Borrow).
3. ¿Cómo reduce el sumador con anticipación de acarreo (Carry Lookahead) el tiempo de retraso de propagación en ALUs grandes de 32 o 64 bits? Detalla el coste de transistores asociado.

Tema 7: Elementos de Memoria Básicos: Biestables

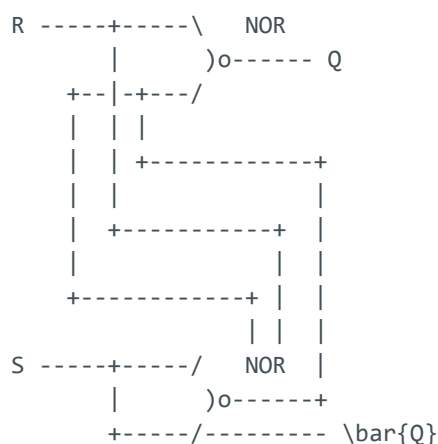
Los sistemas combinacionales carecen de la capacidad de almacenar estados históricos: sus salidas responden de forma ciega e instantánea a las entradas presentes. Para construir computadores que sigan secuencias de instrucciones (programas), necesitamos circuitos capaces de memorizar información. Esto se logra mediante la **realimentación**, donde la salida de una compuerta lógica se conecta de vuelta a su entrada, creando elementos estables de memoria llamados **Latches** y **Biestables (Flip-Flops)**.

1. Realimentación y Latches Asíncronos

Un circuito asíncrono no utiliza señal de reloj. Su estado cambia en el instante en que cambian sus entradas.

Latch RS (Reset-Set) con puertas NOR

Es el elemento de memoria elemental. Consta de dos compuertas NOR realimentadas de forma cruzada:



- **Comportamiento:**

- $S = 1, R = 0 \Rightarrow$ **Set:** La salida pasa a $Q = 1$ (almacena un "1").
- $S = 0, R = 1 \Rightarrow$ **Reset:** La salida pasa a $Q = 0$ (almacena un "0").
- $S = 0, R = 0 \Rightarrow$ **Memoria:** Conserva el estado anterior ($Q_{next} = Q$).
- $S = 1, R = 1 \Rightarrow$ **Estado Prohibido:** Ambas salidas Q y $\bar{Q}$ intentan ponerse a 0, rompiendo la complementariedad. Si las entradas vuelven a 0 a la vez, el circuito entra en una **condición de carrera (race condition)** oscilatoria indeterminada.

2. Biestables Síncronos (Flip-Flops)

Para coordinar millones de celdas de memoria de forma armoniosa y evitar condiciones de carrera, se introduce una señal periódica común de sincronización llamada **Reloj (CLK - Clock)**.

- **Latch:** Es sensible al **nivel** de la señal de habilitación (conduce mientras la señal esté alta).
- **Flip-Flop:** Es sensible únicamente al **flanco (transición rápida)** del reloj (flanco de subida $\uparrow$ o de bajada $\downarrow$). Durante los periodos lógicos estables del reloj, el circuito ignora cualquier cambio en sus entradas.

3. Tipos de Flip-Flops y sus Ecuaciones Características

Tipo	Ecuación Característica	Comportamiento en Flanco Activo	Uso Principal
D (Data)	$Q_{n+1} = D$	La salida toma directamente el valor de la entrada D .	Registros de almacenamiento, buses.
JK	$Q_{n+1} = J\bar{Q}_n + \bar{K}Q_n$	Resuelve el estado prohibido de RS. Si $J = K = 1$, la salida conmuta (inversión).	Contadores, divisores de frecuencia.
T (Toggle)	$Q_{n+1} = T \oplus Q_n$	Si $T = 1$, la salida cambia al estado opuesto; si $T = 0$, se mantiene.	Contadores síncronos, acumuladores.

4. El Toque Informático

SRAM vs. DRAM: La batalla de la Caché y la Memoria Principal

La memoria de un computador se diseña con tecnologías de almacenamiento radicalmente distintas:

1. **SRAM (Static RAM):** Cada celda de memoria consiste en un **latch realimentado (usando de 4 a 6 transistores)**.

- *Ventaja*: Es extremadamente rápida (latencias de menos de 1 ns) porque el estado se mantiene de forma estática y estable mientras reciba corriente.
- *Uso*: Se emplea en la memoria caché integrada de la CPU ($L1, L2, L3$). Su coste de fabricación y área física es muy alto.

2. DRAM (Dynamic RAM): Cada celda consta de **un solo transistor y un condensador minúsculo**.

- *Ventaja*: Altísima densidad de integración y coste bajísimo (permite fabricar módulos de 16 GB de RAM de forma barata).
- *Inconveniente*: El condensador pierde su carga eléctrica en milisegundos. La CPU o el controlador de memoria deben realizar **ciclos de refresco constantes** para leer la celda y recargar el condensador, lo que hace que la DRAM sea mucho más lenta y consume más energía en reposo.

A continuación, implementamos en Python una simulación orientada a objetos de un Flip-Flop D sensible a flanco de subida, mostrando cómo almacena la información sólo en las transiciones de reloj.

```
class FlipFlopD:
    def __init__(self):
        self.Q = 0
        self.clk_prev = 0

    def procesar(self, D, CLK):
        # Detectamos flanco de subida: CLK cambia de 0 a 1
        if self.clk_prev == 0 and CLK == 1:
            print(f"[FLANCO SUBIDA] Actualizando Q: {self.Q} -> {D}")
            self.Q = D
        else:
            # En cualquier otro estado de reloj, la salida se mantiene constante
            pass
        self.clk_prev = CLK
        return self.Q

# Simulación de un cronograma
ff = FlipFlopD()
entradas_cronograma = [
    # (D, CLK)
    (1, 0), # CLK=0, D=1 -> No cambia
    (1, 1), # CLK=1 (Flanco de subida), D=1 -> Q pasa a 1
    (0, 1), # CLK=1 (Nivel alto), D cambia a 0 -> Ignorado (no hay flanco)
    (0, 0), # CLK=0 (Flanco de bajada) -> Ignorado
    (0, 1), # CLK=1 (Flanco de subida), D=0 -> Q pasa a 0
]

print("Simulación de cronograma de Flip-Flop D:")
for idx, (d, clk) in enumerate(entradas_cronograma):
    q = ff.procesar(d, clk)
    print(f" Paso {idx}: D={d}, CLK={clk} -> Salida Q={q}")
```

5. Ejercicios Resueltos

Ejercicio 1

Deducir la ecuación característica de un Flip-Flop JK a partir de su tabla de excitación.

Solución:

1. **Construimos la tabla de transición de estados:**

J	K	Q_n	Q_{n+1}	Minterm asociado
0	0	0	0	-
0	0	1	1	$m_1 = \bar{J}\bar{K}Q_n$
0	1	0	0	-
0	1	1	0	-
1	0	0	1	$m_4 = J\bar{K}\bar{Q}_n$
1	0	1	1	$m_5 = J\bar{K}Q_n$
1	1	0	1	$m_6 = JK\bar{Q}_n$
1	1	1	0	-

2. **Escribimos la ecuación para Q_{n+1} sumando los minterms:**

$$Q_{n+1} = \bar{J}\bar{K}Q_n + J\bar{K}\bar{Q}_n + J\bar{K}Q_n + JK\bar{Q}_n$$

3. **Simplificamos algebraicamente:**

- Agrupamos los términos con factor común $J\bar{Q}_n$:

$$J\bar{Q}_n(\bar{K} + K) = J\bar{Q}_n$$

- Agrupamos los términos con factor común $\bar{K}Q_n$:

$$\bar{K}Q_n(\bar{J} + J) = \bar{K}Q_n$$

- Sumamos los resultados obtenidos:

$$Q_{n+1} = J\bar{Q}_n + \bar{K}Q_n$$

Queda demostrada la ecuación característica del Flip-Flop JK.

Ejercicio 2

Dibujar a mano el cronograma de salida de un Flip-Flop D disparado por flanco de subida, dados las siguientes señales de reloj y datos:

- CLK: pulsos regulares en periodos $t = 1$ (subida), $t = 2$ (bajada), $t = 3$ (subida), $t = 4$ (bajada).
- D: vale 1 en el intervalo $[0, 2]$, y pasa a 0 en el intervalo $[2, 5]$.
- Asumir estado inicial $Q = 0$.

Solución: Analizamos el estado únicamente en los instantes de **flanco de subida** ($t = 1$ y $t = 3$):

1. En $t = 1$ ($\uparrow$ CLK): La entrada D vale 1. La salida Q se actualiza al valor de D , pasando a $Q = 1$.
2. En el intervalo $t \in (1, 3)$: El reloj baja en $t = 2$ e ignoramos el cambio de D a 0 en ese mismo instante. La salida mantiene su valor estable $Q = 1$.
3. En $t = 3$ ($\uparrow$ CLK): La entrada D vale 0. La salida Q se actualiza pasando a $Q = 0$.
4. En el intervalo $t > 3$: La salida mantiene su valor $Q = 0$.

El cronograma final de $Q(t)$ es un pulso que vale 0 para $t \in [0, 1]$, pasa a 1 para $t \in [1, 3]$ y vuelve a 0 para $t > 3$.

6. Ejercicios Propuestos

1. Dibuja el esquema lógico de un Latch RS utilizando compuertas lógicas NAND elementales y deduce su tabla de verdad (indicando cuál es el estado prohibido).
2. Dibuja un diagrama de bloques mostrando cómo construir un Flip-Flop de tipo T utilizando únicamente un Flip-Flop de tipo JK.
3. Explica conceptualmente qué es el **tiempo de establecimiento (setup time)** y el **tiempo de mantenimiento (hold time)** en un biestable síncrono, y analiza por qué violar estos límites físicos provoca inestabilidad (metaestabilidad) en el circuito.

Tema 8: Sistemas Secuenciales: Análisis y Diseño de Máquinas de Estados

Un sistema secuencial síncrono basa su comportamiento en una **Máquina de Estados Finitos (FSM - Finite State Machine)**. A diferencia de los circuitos combinacionales, las salidas de una FSM dependen de sus entradas y del **estado interno** acumulado en sus biestables. Modelar sistemas mediante FSMs es la metodología estándar para diseñar controladores complejos en hardware, incluyendo la Unidad de Control de los propios procesadores.

1. Modelos de Máquinas de Estados Finitos

Existen dos modelos clásicos de FSM que difieren en la forma en que generan sus salidas:

1.1 Modelo de Moore

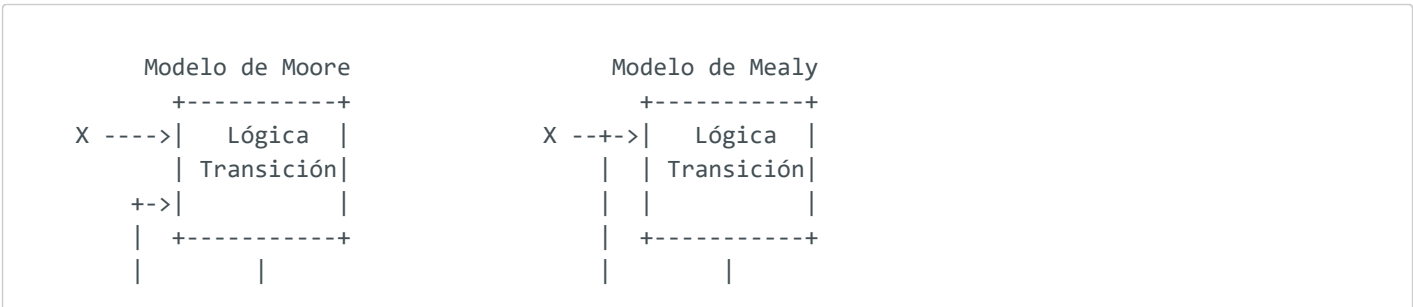
Las salidas dependen **únicamente del estado actual** del sistema.

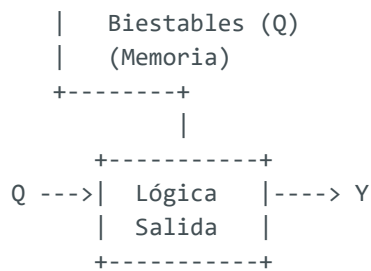
- **Fórmula:** $Y(t) = f(Q(t))$
- **Representación gráfica:** El valor de la salida se dibuja dentro del propio círculo que representa al estado (ej. Estado A / Salida 0).
- **Ventaja:** Las salidas cambian de forma síncrona y limpia en los flancos de reloj, libre de ruidos transitorios (glitches) de entrada.

1.2 Modelo de Mealy

Las salidas dependen del **estado actual y de las entradas presentes** en ese instante.

- **Fórmula:** $Y(t) = f(Q(t), X(t))$
- **Representación gráfica:** El valor de la salida se dibuja en la flecha de transición de estado junto con el valor de la entrada que la provoca (ej. Entrada / Salida).
- **Ventaja:** Suele requerir menos estados para realizar la misma función lógica que una máquina de Moore.





2. Metodología de Diseño de una FSM Síncrona

El proceso para diseñar físicamente un circuito secuencial consta de los siguientes pasos ordenados:

1. **Diagrama de Estados:** Dibujar los estados (círculos) y las transiciones (flechas) que describen el comportamiento del controlador ante las entradas.
2. **Tabla de Transición de Estados:** Traducir el diagrama a una tabla que asocie el Estado Actual (Q_n) y las Entradas (X) con el Estado Siguiendo (Q_{n+1}) y las Salidas (Y).
3. **Codificación de Estados:** Asignar combinaciones binarias a cada estado. Si la máquina tiene M estados, necesitaremos k flip-flops de forma que:

$$2^k \geq M$$

4. **Selección de Flip-Flops:** Usar la **tabla de excitación** del biestable elegido (generalmente D o JK) para determinar qué entradas del flip-flop se necesitan para forzar el salto de Q_n a Q_{n+1} .
5. **Simplificación por Karnaugh:** Obtener las expresiones booleanas simplificadas para las entradas de excitación de los flip-flops y las salidas.
6. **Esquema Lógico:** Dibujar el circuito interconectando compuertas combinacionales y los flip-flops.

3. El Toque Informático

La Unidad de Control de la CPU como una FSM Gigante

En la arquitectura de un procesador, la **Unidad de Control (UC)** actúa como el director de orquesta del computador.

- La UC es conceptualmente una gran FSM síncrona.

- Su **estado** representa la fase actual del ciclo de instrucción (Fetch, Decode, Execute, etc.).
- Su **entrada** principal es el código de operación (OPCode) de la instrucción cargada en el Registro de Instrucción (IR).
- Sus **salidas** son decenas de señales lógicas de control que abren o cierran puertas lógicas en los buses del procesador, habilitan la escritura en registros específicos o le dicen a la ALU si debe sumar o restar.

A continuación, implementamos en Python una simulación lógica de una FSM de Moore diseñada para detectar la secuencia binaria "101" en una transmisión de datos serie.

```
class DetectorSecuencia101:
    def __init__(self):
        # Estados: S0 (reposo), S1 (detectado '1'), S2 (detectado '10'), S3 (detectado '101')
        self.estado = "S0"

    def transicionar(self, bit_entrada):
        # FSM de Moore: la salida Y depende únicamente del estado actual
        # S0 -> Y=0, S1 -> Y=0, S2 -> Y=0, S3 -> Y=1

        # 1. Lógica de transición de estados
        if self.estado == "S0":
            self.estado = "S1" if bit_entrada == 1 else "S0"
        elif self.estado == "S1":
            self.estado = "S2" if bit_entrada == 0 else "S1"
        elif self.estado == "S2":
            self.estado = "S3" if bit_entrada == 1 else "S0"
        elif self.estado == "S3":
            # Si recibimos un 0 tras detectar '101', volvemos a S2 (pues el '1' final de '101'
            # es el '1' inicial del siguiente)
            self.estado = "S2" if bit_entrada == 0 else "S1"

        # 2. Lógica de salida de Moore
        salida = 1 if self.estado == "S3" else 0
        return self.estado, salida

# Simulación con un flujo de bits
secuencia_entrada = [1, 0, 1, 0, 1, 1, 0, 1]
fsm = DetectorSecuencia101()

print("Simulación de FSM detectora de '101':")
for bit in secuencia_entrada:
    est, y = fsm.transicionar(bit)
    print(f"  Entrada: {bit} -> Transiciona a Estado: {est} -> Salida Y: {y}")
```

4. Ejercicios Resueltos

Ejercicio 1

Diseñar una máquina de Moore síncrona utilizando Flip-Flops D que detecte la secuencia "11" en un flujo de bits serie y active una salida $Y = 1$.

Solución:

1. Definir estados:

- A : Estado de reposo (no se han recibido '1's). Salida $Y = 0$.
- B : Se ha recibido un único '1'. Salida $Y = 0$.
- C : Se han recibido dos o más '1's consecutivos (secuencia detectada). Salida $Y = 1$.

2. Codificación de estados (3 estados $\Rightarrow$ 2 bits, Flip-Flops Q_1, Q_0):

- $A = 00_2$
- $B = 01_2$
- $C = 11_2$

3. Construir la Tabla de Transición y Excitación: Como usamos Flip-Flops D, la entrada D_i es exactamente igual al estado siguiente deseado $Q_{i,next}$:

Q_1Q_0	Entrada X	$Q_{1,next}Q_{0,next}$	Entradas D_1D_0	Salida Y
00 (A)	0	00 (A)	00	0
00 (A)	1	01 (B)	01	0
01 (B)	0	00 (A)	00	0
01 (B)	1	11 (C)	11	0
11 (C)	0	00 (A)	00	1
11 (C)	1	11 (C)	11	1

4. Simplificar ecuaciones por Karnaugh:

- **Ecuación de Salida Y** : Depende solo del estado actual. Inspeccionando la tabla:

$$Y = Q_1Q_0$$

- **Ecuación para D_0** :

$$D_0 = \bar{Q}_1\bar{Q}_0X + \bar{Q}_1Q_0X + Q_1Q_0X = X(\bar{Q}_1\bar{Q}_0 + \bar{Q}_1Q_0 + Q_1Q_0) = X(Q_0 + \bar{Q}_1)$$

- **Ecuación para D_1** :

$$D_1 = \bar{Q}_1Q_0X + Q_1Q_0X = Q_0X(\bar{Q}_1 + Q_1) = Q_0X$$

El circuito final utiliza 2 Flip-Flops D con entradas $D_0 = X(Q_0 + \bar{Q}_1)$ y $D_1 = Q_0X$, y una puerta AND para la salida $Y = Q_1Q_0$.

5. Ejercicios Propuestos

1. Dibuja el diagrama de transición de estados de una máquina de Mealy que reciba un bit serie de entrada y active su salida a 1 únicamente en el instante en que detecte un flanco de cambio de bits (secuencia "01" o "10").
2. Explica conceptualmente la diferencia física y de cronograma entre una salida en el modelo de Mealy y una salida en el modelo de Moore. ¿A qué se refiere el término **riesgo de transición (hazards / glitches)** en Mealy?
3. Diseña la tabla de transición de estados de un contador síncrono Gray de 2 bits utilizando Flip-Flops D (secuencia de estados: $00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00 \dots$).

Tema 9: Aplicaciones Secuenciales: Registros y Contadores

Para procesar y mover datos de forma agregada (en bloques de 8, 16, 32 o 64 bits), los flip-flops individuales se agrupan formando estructuras secuenciales de propósito general. Las dos aplicaciones más importantes en la arquitectura de computadores son los **Registros de Desplazamiento** (utilizados para conversión y transmisión de datos) y los **Contadores** (utilizados para secuenciar operaciones y controlar el tiempo).

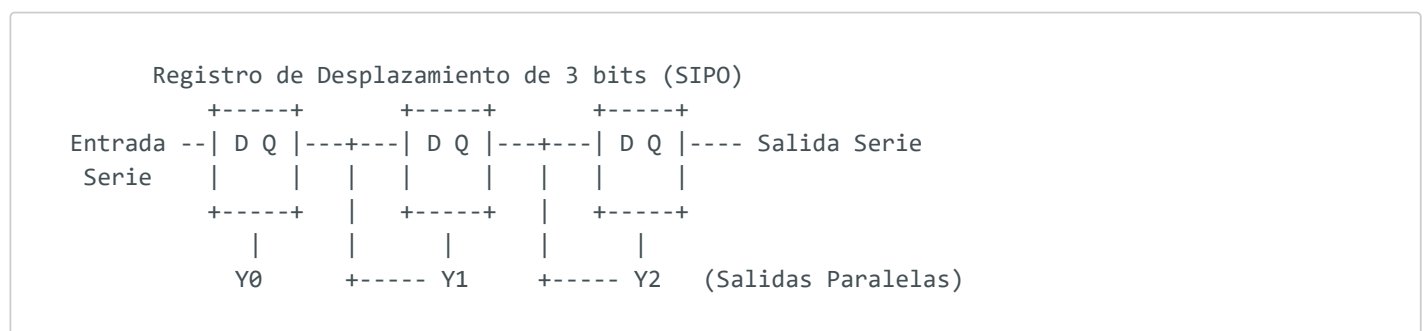
1. Registros de Almacenamiento y Desplazamiento

1.1 Registros de Almacenamiento

Consiste simplemente en N Flip-Flops de tipo D compartiendo la misma señal de reloj (CLK) y una línea de habilitación común ($Enable$). Sirven para retener una palabra de datos binaria de forma temporal.

1.2 Registros de Desplazamiento (Shift Registers)

Son circuitos secuenciales donde los flip-flops se conectan en cascada, de forma que la salida de cada biestable es la entrada del siguiente. En cada flanco de reloj, la información se desplaza una posición a lo largo de la cadena.



Dependiendo de cómo se introduzcan y lean los datos, se clasifican en:

- **SISO (Serial-In, Serial-Out):** Entrada serie, salida serie.
- **SIPO (Serial-In, Parallel-Out):** Entrada serie, salidas paralelas.
- **PISO (Parallel-In, Serial-Out):** Entradas paralelas, salida serie.
- **PIPO (Parallel-In, Parallel-Out):** Entradas y salidas paralelas (el registro básico).

Operación Aritmética por Desplazamiento

Los registros de desplazamiento son multiplicadores y divisores de altísima velocidad:

- Desplazar un número binario **una posición a la izquierda** equivale a **multiplicarlo por 2**:

$$00001010_2(10_{10}) \ll 1 \Rightarrow 00010100_2(20_{10})$$

- Desplazar un número binario **una posición a la derecha** equivale a realizar una **división entera por 2**:

$$00001010_2(10_{10}) \gg 1 \Rightarrow 00000101_2(5_{10})$$

2. Contadores

Un contador es un circuito secuencial que recorre una secuencia predeterminada de estados en respuesta a los pulsos de reloj. El número de estados distintos de la secuencia es el **Módulo (M)** del contador.

2.1 Contadores Asíncronos (Contadores de Rizado)

El reloj solo se conecta al primer flip-flop. Las etapas posteriores se disparan utilizando como señal de reloj la salida de la etapa anterior.

- *Inconveniente*: Se acumulan los retardos de conmutación de cada etapa ($O(N)$). Durante el cambio, se producen estados intermedios transitorios erróneos en las salidas. No son recomendables para altas frecuencias.

2.2 Contadores Síncronos

Todos los flip-flops comparten exactamente la misma línea de reloj, por lo que conmutan simultáneamente en el mismo flanco. Para determinar cuándo debe cambiar cada etapa, se diseña una red lógica combinacional conectada a las entradas de control (J, K o T) de los flip-flops.

3. El Toque Informático

La UART y los Puertos USB/COM

Los microprocesadores procesan la información de forma **paralela** en buses anchos (por ejemplo, leyendo 64 bits a la vez en su memoria caché). Sin embargo, enviar 64 cables físicos a través de una red o a un periférico externo (como una impresora o un disco externo) es físicamente costoso y produce interferencias electromagnéticas destructivas.

- Para solucionarlo se utiliza un chip llamado **UART (Universal Asynchronous Receiver-Transmitter)**.
- La UART toma la palabra paralela del bus del sistema, la carga en un registro de desplazamiento de tipo **PISO** y la transmite bit a bit en serie a través de un único cable (como el puerto USB).
- En el receptor, otra UART recoge los bits serie mediante un registro de desplazamiento **SIPO**, reconstruye la palabra paralela original y la introduce en el bus del computador destino.

A continuación, implementamos en Python una simulación lógica de un registro de desplazamiento de 8 bits que demuestra el desplazamiento aritmético de multiplicación y división binaria.

```
class RegistroDesplazamiento8Bits:
    def __init__(self, valor_inicial=0):
        # Almacenamos el registro como una lista de 8 bits (MSB a la izquierda)
        self.bits = [(valor_inicial >> i) & 1 for i in reversed(range(8))]

    def shift_left(self, bit_entrada=0):
        # Desplazamiento a la izquierda (Multiplicación por 2)
        # El bit que sale a la izquierda (MSB) se descarta, el bit_entrada entra por la derecha
        (LSB)
        self.bits.pop(0)
        self.bits.append(bit_entrada)
        return self.obtener_valor()

    def shift_right(self, bit_entrada=0):
        # Desplazamiento a la derecha (División por 2)
        # El bit que sale a la derecha (LSB) se descarta, el bit_entrada entra por la izquierda
        (MSB)
        self.bits.pop()
        self.bits.insert(0, bit_entrada)
        return self.obtener_valor()

    def obtener_valor(self):
        val = 0
        for b in self.bits:
            val = (val << 1) | b
        return val

    def __str__(self):
        return "".join(map(str, self.bits))

# Simulación
reg = RegistroDesplazamiento8Bits(10) # Inicializamos con 10 decimal (00001010)
print(f"Estado inicial:      {reg} (Decimal: {reg.obtener_valor()})")

reg.shift_left(0)
print(f"Desplazamiento Izq:   {reg} (Decimal: {reg.obtener_valor()}) -> Multiplicado por 2")

reg.shift_right(0)
```

```
reg.shift_right(0)
print(f"Dos Desplazamientos Der: {reg} (Decimal: {reg.obtener_valor()}) -> Dividido por 4")
```

4. Ejercicios Resueltos

Ejercicio 1

Diseñar un contador síncrono Gray de 2 bits ($00 \rightarrow 01 \rightarrow 11 \rightarrow 10 \rightarrow 00 \dots$) utilizando Flip-Flops D.

Solución:

1. Tabla de Transición (Siguiendo Estado):

Estado Actual Q_1Q_0	Estado Siguiente $Q_{1,next}Q_{0,next}$	Entradas D_1D_0
00	01	01
01	11	11
11	10	10
10	00	00

2. Deducir ecuaciones lógicas para las entradas D_1 y D_0 :

- Para D_1 (filas con salida 1): se activa para $Q_1Q_0 = 01$ y $Q_1Q_0 = 11$.

$$D_1 = \bar{Q}_1Q_0 + Q_1Q_0 = Q_0(\bar{Q}_1 + Q_1) = Q_0$$

- Para D_0 (filas con salida 1): se activa para $Q_1Q_0 = 00$ y $Q_1Q_0 = 01$.

$$D_0 = \bar{Q}_1\bar{Q}_0 + \bar{Q}_1Q_0 = \bar{Q}_1(\bar{Q}_0 + Q_0) = \bar{Q}_1$$

3. Resultado: El circuito consta de 2 Flip-Flops D:

- Conectamos la salida Q_0 del segundo flip-flop directamente a la entrada D_1 del primero.
- Conectamos la salida negada $\bar{Q}_1$ del primer flip-flop a la entrada D_0 del segundo.

El circuito recorrerá la secuencia Gray síncronamente al ritmo del reloj.

Ejercicio 2

Explicar el funcionamiento de un contador Johnson de 3 bits y listar la secuencia de estados que recorre si parte del estado inicial 000_2 .

Solución: Un contador Johnson se construye conectando la salida negada del último flip-flop a la entrada D del primer flip-flop de un registro de desplazamiento:

$$D_0 = \bar{Q}_2, \quad D_1 = Q_0, \quad D_2 = Q_1$$

1. Estado inicial: 000 (salida negada del último es 1).
2. Flanco 1: entra 1 $\Rightarrow$ 100.
3. Flanco 2: entra 1 $\Rightarrow$ 110.
4. Flanco 3: entra 1 $\Rightarrow$ 111 (ahora la salida del último es 1, por lo que entra su negada 0).
5. Flanco 4: entra 0 $\Rightarrow$ 011.
6. Flanco 5: entra 0 $\Rightarrow$ 001.
7. Flanco 6: entra 0 $\Rightarrow$ 000 (vuelve al estado inicial).

La secuencia consta de 6 estados distintos ($2N$ estados para N bits).

5. Ejercicios Propuestos

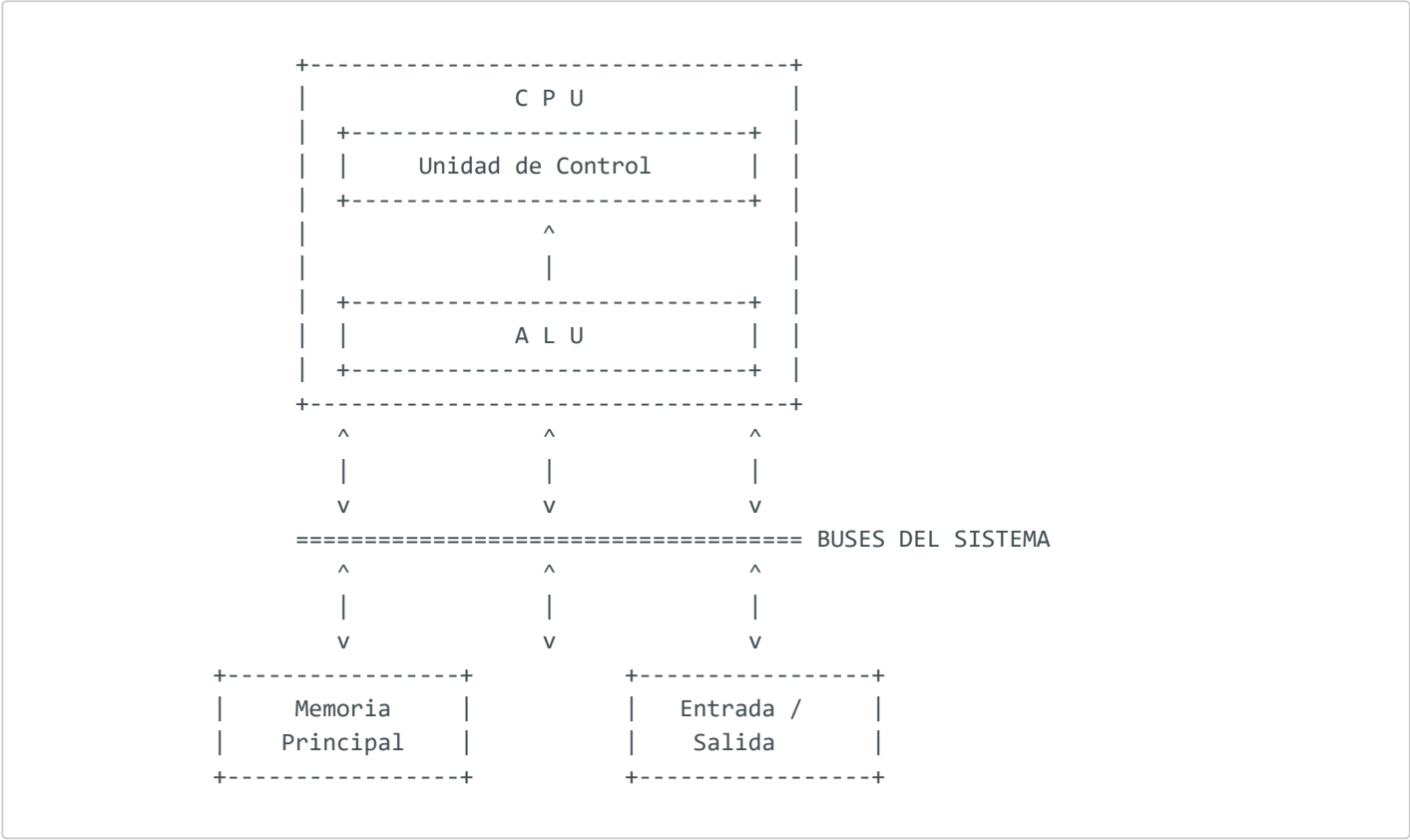
1. Dibuja el esquema lógico de un contador asíncrono binario de 3 bits utilizando Flip-Flops JK configurados en modo conmutación ($J = K = 1$).
2. Diseña la lógica de reinicio (Reset) para convertir un contador binario síncrono de 4 bits en un contador de módulo 10 (contador BCD, de 0 a 9).
3. Explica cómo realiza un registro de desplazamiento bidireccional la selección entre desplazamiento a la izquierda y a la derecha utilizando multiplexores en sus entradas D .

Tema 10: Estructura Interna del Computador: El Modelo de Von Neumann

Hasta mediados de la década de 1940, los primeros computadores eran máquinas de propósito único: para cambiar de programa (por ejemplo, pasar de calcular trayectorias balísticas a resolver sistemas de ecuaciones), los operadores debían reconfigurar físicamente los cables e interruptores de la máquina (programación por hardware). El matemático **John von Neumann** introdujo la revolucionaria idea del **Programa Almacenado**, donde las instrucciones del programa se codifican numéricamente y se almacenan en la misma memoria física que los datos, dando origen a la arquitectura universal de los computadores modernos.

1. Los Subsistemas Principales del Modelo de Von Neumann

La arquitectura de Von Neumann divide el computador en tres bloques funcionales bien diferenciados interconectados entre sí:



1. **Unidad Central de Procesamiento (CPU):**
- **Unidad de Control (UC):** Interpreta y secuencia las instrucciones del programa almacenado.
 - **Unidad Aritmético-Lógica (ALU):** Realiza los cálculos lógicos y aritméticos.

- **Banco de Registros:** Celdas de memoria internas ultrarrápidas para almacenar operandos inmediatos.
2. **Memoria Principal:** Un array de celdas direccionables linealmente en el que conviven de forma indistinta los datos y las instrucciones de los programas.
 3. **Subsistema de Entrada/Salida (E/S):** Interfaz para comunicar el computador con periféricos externos (teclados, monitores, discos).
-

2. Los Buses del Sistema

Los subsistemas se comunican mediante tres buses (conjuntos de pistas de cobre paralelas):

- **Bus de Datos (Bidireccional):** Transporta los datos de las operaciones y los códigos de instrucción leídos de la memoria. Su ancho (ej. 32 o 64 bits) define la longitud de palabra nativa del procesador.
 - **Bus de Direcciones (Unidireccional):** La CPU coloca en este bus la dirección de memoria física exacta a la que desea acceder. Solo la CPU escribe en este bus. Su ancho determina el espacio de direccionamiento máximo del procesador (por ejemplo, con 32 líneas de dirección, el procesador puede direccionar hasta $2^{32} = 4 \text{ GB}$ de RAM).
 - **Bus de Control (Bidireccional):** Transporta señales de temporización y sincronización (línea de lectura/escritura, señales de interrupción, reloj del sistema).
-

3. Arquitectura Von Neumann frente a Harvard

- **Arquitectura Von Neumann:** Datos e instrucciones comparten el **mismo bus físico y la misma memoria**.
 - **Arquitectura Harvard:** Dispone de **dos memorias físicas separadas** (Memoria de Datos y Memoria de Instrucciones), cada una con su respectivo bus independiente.
-

4. El Toque Informático

El Cuello de Botella de Von Neumann y la Solución Caché L1

En el modelo Von Neumann puro, la CPU no puede leer una instrucción (operación Fetch) y escribir o leer un dato en memoria (operación Execute) simultáneamente, ya que comparten el

mismo bus físico. Este retardo se conoce como el **Cuello de Botella de Von Neumann (Von Neumann Bottleneck)** y limita gravemente el rendimiento en procesadores rápidos.

- **La Solución en Microchips Modernos:** Los procesadores actuales (Intel Core, AMD Ryzen, Apple M1) emplean un diseño híbrido:
 - Hacia el **interior de la CPU (en el núcleo)**: Se utiliza una arquitectura **tipo Harvard** integrando dos memorias caché independientes de Nivel 1: la **Caché L1 de Instrucciones (L1I)** y la **Caché L1 de Datos (L1D)**, permitiendo accesos concurrentes de máxima velocidad.
 - Hacia el **exterior de la CPU (placa base)**: Se utiliza una arquitectura **tipo Von Neumann** unificada en la memoria RAM principal por simplicidad física de cableado.

A continuación, implementamos en Python una simulación lógica del comportamiento de los buses de direcciones y datos al acceder a una memoria unificada (tipo Von Neumann).

```
class MemoriaVonNeumann:
    def __init__(self, tamano=16):
        # Memoria de 16 posiciones. Almacena de forma indistinta datos e instrucciones.
        # Las instrucciones se representan como diccionarios, los datos como enteros
        self.celdas = [0] * tamano

    def escribir(self, bus_direcciones, bus_datos):
        # Simulación de escritura controlada por bus de control
        self.celdas[bus_direcciones] = bus_datos
        print(f"[ESCRITURA] Celda {bus_direcciones:02d} <- Guardado: {bus_datos}")

    def leer(self, bus_direcciones):
        # Simulación de lectura colocando el dato de la celda en el bus de datos
        bus_datos = self.celdas[bus_direcciones]
        print(f"[LECTURA] Bus Direcciones colocó: {bus_direcciones:02d} -> Bus Datos lee: {bus_datos}")
        return bus_datos

# Simulación de carga y ejecución de un programa
mem = MemoriaVonNeumann(16)

# Cargamos el programa en las posiciones 0 y 1 (Instrucciones)
mem.escribir(0, {"op": "ADD", "addr": 5}) # Instrucción: Suma el dato de la dirección 5
mem.escribir(1, {"op": "SUB", "addr": 6}) # Instrucción: Resta el dato de la dirección 6

# Cargamos los datos en las direcciones 5 y 6
mem.escribir(5, 120) # Dato 1 = 120
mem.escribir(6, 45) # Dato 2 = 45

print("\n--- Ejecución del Procesador (Ciclo Fetch-Execute) ---")
# 1. Fetch de la primera instrucción (en dirección 0)
instruccion1 = mem.leer(0)
# 2. Execute (requiere leer el dato de la dirección de memoria indicada por la instrucción)
dato = mem.leer(instruccion1["addr"])
print(f" CPU ejecuta: {instruccion1['op']} con el valor {dato}")
```

5. Ejercicios Resueltos

Ejercicio 1

Un microcontrolador tiene un bus de direcciones de 16 líneas y un bus de datos de 8 líneas. Calcular:

1. El tamaño máximo de memoria física direccionable en bytes.
2. El tamaño de la palabra del computador.

Solución:

1. **Cálculo de la capacidad de direccionamiento:** El bus de direcciones consta de 16 bits. El número de combinaciones de direcciones físicas únicas es:

$$\text{Direcciones} = 2^{16} = 65.536 \text{ posiciones}$$

2. **Cálculo de la memoria direccionable:** Dado que el bus de datos tiene 8 líneas, cada palabra direccionable almacena exactamente 8 bits (1 byte):

$$\text{Capacidad} = 65.536 \times 1 \text{ Byte} = 64 \text{ KB}$$

El procesador puede direccionar un mapa de memoria máximo de 64 KB (65.536 bytes).

Ejercicio 2

Explicar por qué la arquitectura Harvard permite un mayor rendimiento y menor latencia en comparación con el modelo Von Neumann a coste de un diseño físico más complejo.

Solución: En la arquitectura Harvard, al estar la memoria de instrucciones físicamente separada de la memoria de datos y conectadas mediante buses independientes, la CPU puede leer una nueva instrucción de la memoria (fase Fetch) y acceder simultáneamente a la memoria de datos (para leer o escribir operandos en la fase Execute).

- **Rendimiento:** Se dobla el ancho de banda del bus efectivo del sistema, eliminando el "cuello de botella de Von Neumann".
- **Complejidad:** Exige duplicar el número de pines de direcciones y datos de la CPU (aumentando el tamaño del circuito integrado y el número de pistas de cobre en la placa base) y duplica el número de controladores de bus de memoria, aumentando drásticamente el coste físico de fabricación.

6. Ejercicios Propuestos

1. Determina el rango de direccionamiento máximo en bytes para un procesador de 32 bits (cuyo bus de direcciones tiene 32 líneas) y para uno de 64 bits.

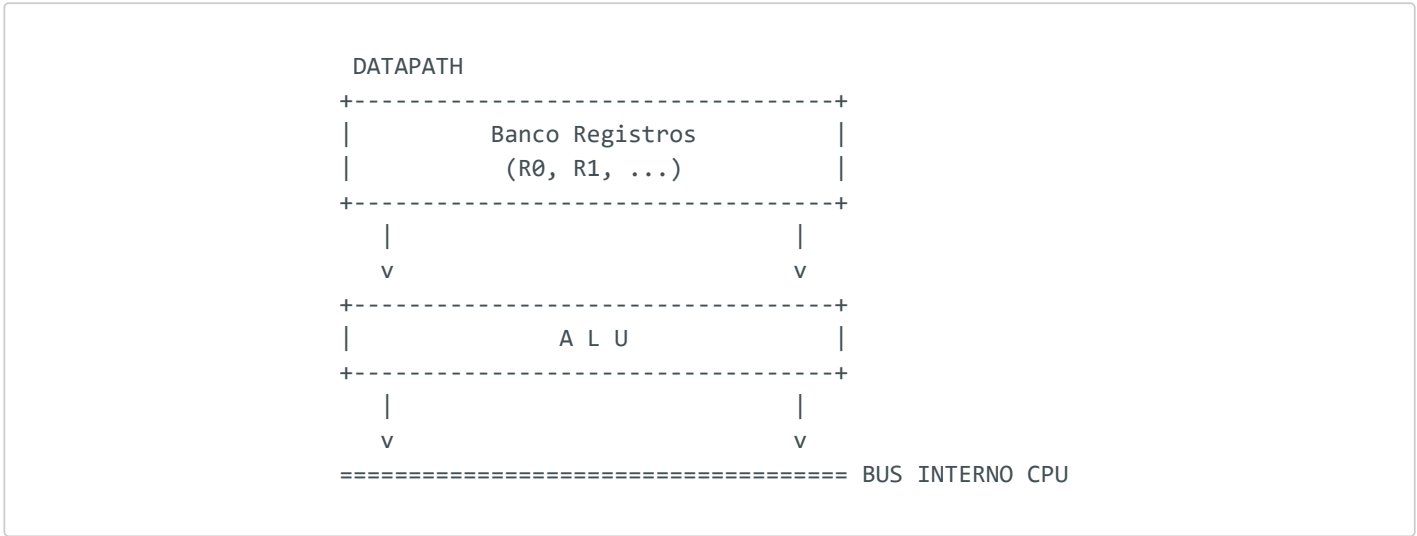
2. Dibuja un diagrama conceptual de bloques que ilustre la diferencia en los buses de conexión en una configuración tipo Von Neumann frente a una configuración Harvard.
3. ¿Qué es la memoria de Entrada/Salida mapeada en memoria (Memory-Mapped I/O) y en qué se diferencia del direccionamiento de Entrada/Salida aislada (Port-Mapped I/O) desde el punto de vista del bus de direcciones?

Tema 11: Organización del Procesador: Ruta de Datos y Unidad de Control

La Unidad Central de Procesamiento (CPU) es el cerebro del computador encargado de buscar, decodificar y ejecutar las instrucciones de los programas. Internamente, la CPU se organiza dividiendo sus funciones en dos grandes estructuras: la **Ruta de Datos (Datapath)**, que actúa como el cuerpo físico (realiza las transferencias y operaciones lógicas sobre los operandos), y la **Unidad de Control (UC)**, que actúa como el cerebro director (secuencia y activa los caminos correctos de la ruta de datos).

1. El Camino de Datos (Datapath) y sus Registros

La Ruta de Datos contiene todos los elementos que almacenan y procesan los operandos numéricos:



- **La ALU:** Ejecuta las operaciones sobre los operandos de entrada.
- **Banco de Registros (Register File):** Conjunto de registros de alta velocidad direccionables por el programador (por ejemplo, registros `$s0`, `$t0` en MIPS) para retener operandos temporales.
- **Registro de Flags:** Almacena las señales de estado (Z, S, V, C) producidas por la última operación de la ALU.

2. Los Registros de Control e Interfaz de Bus

Son registros internos especiales de la CPU, invisibles para el programador de alto nivel, indispensables para orquestar la comunicación con la memoria principal a través del bus del sistema:

1. **Contador de Programa (PC - Program Counter):** Almacena la dirección de memoria de la **siguiente** instrucción que debe leerse de memoria. Se incrementa automáticamente tras cada lectura.
 2. **Registro de Instrucción (IR - Instruction Register):** Retiene el código binario de la instrucción que la CPU está ejecutando en ese momento.
 3. **Registro de Direcciones de Memoria (MAR - Memory Address Register):** Conectado físicamente al bus de direcciones. Almacena la dirección exacta a la que la CPU desea acceder en memoria para leer o escribir.
 4. **Registro de Datos de Memoria (MDR - Memory Data Register):** Conectado físicamente al bus de datos. Actúa como buffer bidireccional; retiene el dato leído de memoria o el dato listo para escribirse en ella.
-

3. La Unidad de Control (UC)

La Unidad de Control interpreta la instrucción guardada en el IR y genera las señales lógicas que controlan el flujo del dato. Existen dos filosofías de diseño de la UC:

3.1 Unidad de Control Cableada

Se implementa utilizando circuitos combinacionales puros (decodificadores, puertas lógicas AND/OR, flip-flops de estado).

- **Ventaja:** Es extremadamente rápida. Minimiza el retardo de conmutación.
- **Inconveniente:** Es muy rígida. Si queremos añadir una nueva instrucción a la CPU, debemos rediseñar físicamente todo el circuito integrado. Típica en arquitecturas RISC (como ARM, MIPS).

3.2 Unidad de Control Microprogramada

Las señales lógicas no se cablean. En su lugar, se almacenan en una memoria ROM de control interna en forma de "microprogramas". Cada instrucción del procesador se traduce en una secuencia de microinstrucciones leídas de la ROM interna.

- **Ventaja:** Es extremadamente flexible. Permite corregir fallos o añadir instrucciones actualizando la microprogramación de la ROM interna.
- **Inconveniente:** Es más lenta por requerir accesos de memoria internos. Típica en arquitecturas complejas CISC (como x86).

4. El Toque Informático

Segmentación del Cauce (Pipelining) en CPUs Modernas

En los microprocesadores comerciales, las instrucciones no se procesan de forma estrictamente secuencial de principio a fin.

- Se utiliza la técnica de **Segmentación (Pipelining)**, similar a una cadena de montaje de automóviles.
- La CPU se divide en etapas independientes (ej. 5 etapas: Fetch, Decode, Execute, Memory, Write-back).
- Mientras una instrucción está ejecutándose en la ALU (etapa Execute), la siguiente instrucción se está decodificando en la etapa Decode, y una tercera se está leyendo de memoria en la etapa Fetch.

Para que esta segmentación funcione sin colisiones, la CPU integra **Registros de Segmentación (Pipeline Registers)** intermedios entre cada etapa. Estos registros retienen de forma síncrona el estado parcial de la instrucción para entregárselo a la siguiente etapa en el flanco de reloj, permitiendo procesar una instrucción por ciclo de reloj de forma efectiva.

A continuación, implementamos en Python una simulación lógica del estado de los registros internos de la CPU (PC, IR, MAR, MDR) durante la carga de un registro.

```
class CPUState:
    def __init__(self):
        # Registros accesibles al programador
        self.registros = {"R0": 0, "R1": 0, "R2": 0}

        # Registros de control internos
        self.PC = 0x1000 # Dirección inicial del programa
        self.IR = None
        self.MAR = 0x0
        self.MDR = 0x0

    def imprimir_estado(self):
        print(f"Estado Registros CPU: PC=0x{self.PC:04X}, IR={self.IR}, MAR=0x{self.MAR:04X}, MDR=0x{self.MDR:04X}")
        print(f" Banco de Registros: R0={self.registros['R0']}, R1={self.registros['R1']}, R2={self.registros['R2']}\n")

# Simulación de CPU
cpu = CPUState()
cpu.imprimir_estado()

# Paso 1: Carga de dirección de instrucción a MAR
cpu.MAR = cpu.PC
print("[CICLO FETCH] Cargando PC a MAR...")
```

```
cpu.imprimir_estado()

# Paso 2: Bus de datos simula lectura de la instrucción de memoria (dirección 0x1000)
# La instrucción es cargar el valor 42 en R0
cpu.MDR = "LD R0, #42"
cpu.IR = cpu.MDR
cpu.PC += 4 # Incrementamos PC (instrucciones de 4 bytes)
print("[CICLO FETCH] Lectura de memoria terminada. Instrucción cargada en IR.")
cpu.imprimir_estado()

# Paso 3: Ejecución (Decode y Execute)
# La CPU ejecuta la instrucción almacenada en el IR
cpu.registros["R0"] = 42
print("[CICLO EXECUTE] Ejecutando: R0 <- 42")
cpu.imprimir_estado()
```

5. Ejercicios Resueltos

Ejercicio 1

Describir detalladamente el flujo de registros de control internos involucrados cuando la CPU realiza la fase de búsqueda (Fetch) de una nueva instrucción de memoria.

Solución: El flujo de registros durante el ciclo Fetch se realiza siguiendo esta secuencia de pasos:

1. **MAR $\leftarrow$ PC:** La dirección contenida en el Contador de Programa (PC) se transfiere al Registro de Direcciones de Memoria (MAR). Esta dirección se coloca en el bus de direcciones.
2. **Lectura de Memoria:** La Unidad de Control activa la señal de lectura (Read) en el bus de control. La memoria responde localizando la dirección e introduciendo el código binario de la instrucción en el bus de datos.
3. **MDR $\leftarrow$ Bus de Datos:** El Registro de Datos de Memoria (MDR) captura la instrucción del bus de datos.
4. **IR $\leftarrow$ MDR:** El código de la instrucción se transfiere desde el MDR al Registro de Instrucción (IR), donde queda retenido para su decodificación.
5. **PC $\leftarrow$ PC + valor:** El Contador de Programa (PC) se incrementa para apuntar a la dirección física de la siguiente instrucción en memoria (típicamente se le suma 1, 2 o 4 según el tamaño de la instrucción en bytes).

Ejercicio 2

Comparar la Unidad de Control Cableada y la Microprogramada en base a su velocidad de reloj máxima y la facilidad para actualizar el repertorio de instrucciones (ISA).

Solución:

1. Velocidad de reloj máxima:

- *Cableada*: **Superior**. Al estar implementada con compuertas combinacionales optimizadas a nivel de silicio, el retardo de conmutación de señales es mínimo, permitiendo frecuencias de reloj muy elevadas.
- *Microprogramada*: **Inferior**. Cada instrucción de máquina exige realizar accesos de lectura adicionales a una memoria ROM de control interna (para extraer las microinstrucciones), lo que añade ciclos de reloj y limita la frecuencia máxima.

2. Facilidad de actualización:

- *Cableada*: **Nula**. Cualquier modificación o añadido en el repertorio de instrucciones exige cambiar físicamente el diseño del silicio y fabricar un chip nuevo.
- *Microprogramada*: **Alta**. El repertorio se puede modificar o ampliar simplemente reescribiendo la ROM de control interna (firmware del microprocesador), sin alterar los transistores del datapath.

6. Ejercicios Propuestos

1. Dibuja un diagrama esquemático mostrando las conexiones de los buses externos de direcciones y datos con los registros MAR y MDR de la CPU.
2. Explica la diferencia entre un registro de propósito general accesible por el programador (como los del Register File) y un registro de control (como el IR o MAR) detallando sus funciones en la CPU.
3. Investiga el término **Riesgos de Control (Control Hazards)** y **Riesgos de Datos (Data Hazards)** en la arquitectura segmentada (pipeline) y cómo afectan al rendimiento de la CPU.

Tema 12: El Ciclo de Instrucción Paso a Paso

El ciclo de instrucción es el proceso secuencial repetitivo mediante el cual un computador extrae una instrucción de la memoria, determina qué acción requiere y la ejecuta. Para comprender a bajo nivel cómo opera el hardware, debemos modelar este ciclo utilizando el **Lenguaje de Transferencia entre Registros (RTL - Register Transfer Language)**, un formalismo matemático que describe el flujo detallado de datos entre los registros de la CPU en cada ciclo de reloj.

1. Fases del Ciclo de Instrucción

Cualquier instrucción de máquina, desde una suma básica hasta un salto condicional, recorre secuencialmente las siguientes fases:



- Búsqueda (Fetch):** Se lee la instrucción apuntada por el PC desde la memoria RAM y se guarda en el Registro de Instrucción (IR).
- Decodificación (Decode):** La Unidad de Control analiza el código binario en el IR para identificar la operación (OPCode) y los modos de direccionamiento de los operandos.

3. **Búsqueda de Operandos:** Si la instrucción requiere datos que residen en memoria, se calculan sus direcciones efectivas y se leen de la RAM.
 4. **Ejecución (Execute):** La ALU realiza la operación indicada (aritmética o lógica).
 5. **Escritura (Write-Back):** El resultado obtenido se escribe en el registro destino del banco de registros o en la memoria RAM.
-

2. Lenguaje de Transferencia entre Registros (RTL)

El RTL modela las microoperaciones temporales que ocurren dentro del procesador. Se utiliza la notación $M[\text{dirección}]$ para referirse a una lectura en la memoria física.

Ejemplo: El Ciclo de Búsqueda (Fetch) en RTL

El ciclo Fetch es universal para todas las instrucciones y requiere tres ciclos de reloj (estados de tiempo T_1, T_2, T_3):

- $T_1 : MAR \leftarrow PC$ (Dirección al bus de direcciones).
 - $T_2 : MDR \leftarrow M[MAR], \quad PC \leftarrow PC + 4$ (Lectura de memoria e incremento simultáneo del PC).
 - $T_3 : IR \leftarrow MDR$ (Carga de la instrucción en el registro de decodificación).
-

3. Flujo en RTL para Instrucciones Típicas

Una vez terminada la fase de Fetch (T_1 a T_3), la Unidad de Control ejecuta las fases restantes en los siguientes ciclos de reloj según la instrucción:

3.1 Instrucción de Carga (Load Word - LW R1, [1000])

Lee el dato de la dirección de memoria 1000 y lo almacena en el registro R1:

- $T_4 : MAR \leftarrow \text{Dirección}(1000)$ (Se envía la dirección del dato).
- $T_5 : MDR \leftarrow M[MAR]$ (Se lee el dato de la memoria).
- $T_6 : R1 \leftarrow MDR$ (El dato se guarda en el banco de registros).

3.2 Instrucción de Suma Aritmética (ADD R1, R2, R3)

Suma el contenido de R2 y R3 y guarda el resultado en R1 (operación puramente interna de la CPU):

- T_4 : Entrada_A_ALU $\leftarrow$ R2, Entrada_B_ALU $\leftarrow$ R3 (Operandos a la ALU).
- T_5 : R1 $\leftarrow$ Salida_ALU (Escritura del resultado en el registro de destino).

4. El Toque Informático

El Reloj del Sistema y el Camino Crítico (Critical Path)

Cada microoperación descrita en RTL debe completarse de forma segura en exactamente un ciclo del reloj del procesador.

- **El Camino Crítico (Critical Path)** es la ruta física de compuertas que tarda más tiempo en propagar sus señales electrónicas (típicamente, el camino que realiza una suma compleja en la ALU y escribe el resultado en memoria).
- **Consecuencia:** La frecuencia de reloj máxima del procesador está limitada por el inverso del retardo del camino crítico:

$$f_{\text{máx}} = \frac{1}{\text{Retardo del camino crítico}}$$

Si intentamos overclockear (subir la frecuencia de reloj) más allá de este límite físico, las señales no llegarán a estabilizarse en los registros antes del siguiente flanco de reloj, provocando corrupción de datos y el temido pantallazo azul del sistema.

A continuación, implementamos en Python una simulación que traza paso a paso los valores de los registros internos (PC, MAR, MDR, IR y Banco de Registros) durante la ejecución de una instrucción de carga de memoria (LW).

```
class SimuladorCicloInstruccion:
    def __init__(self):
        # RAM simulada: instrucciones en direcciones bajas, datos en direcciones altas
        self.ram = {
            0x1000: "LW R1, [0x2000]", # Instrucción en la dirección de PC
            0x2000: 99                # Dato almacenado en memoria
        }
        self.PC = 0x1000
        self.MAR = 0
        self.MDR = 0
```

```

self.IR = ""
self.registros = {"R1": 0}

def trazar_ejecucion(self):
    print(f"Estado Inicial: PC=0x{self.PC:04X}, R1={self.registros['R1']}\n")

    # T1: MAR <- PC
    self.MAR = self.PC
    print(f"T1: MAR <- PC | MAR = 0x{self.MAR:04X}")

    # T2: MDR <- M[MAR] e incrementamos PC
    self.MDR = self.ram[self.MAR]
    self.PC += 4
    print(f"T2: MDR <- M[MAR], PC <- PC+4 | MDR = '{self.MDR}', PC = 0x{self.PC:04X}")

    # T3: IR <- MDR
    self.IR = self.MDR
    print(f"T3: IR <- MDR | IR = '{self.IR}'\n--- Fin de Fetch ---\n")

    # T4 (Decode y Execute de LW R1, [0x2000]): MAR <- Dirección del dato
    direccion_dato = 0x2000
    self.MAR = direccion_dato
    print(f"T4: MAR <- Dirección del dato | MAR = 0x{self.MAR:04X}")

    # T5: MDR <- M[MAR]
    self.MDR = self.ram[self.MAR]
    print(f"T5: MDR <- M[MAR] | MDR = {self.MDR}")

    # T6: R1 <- MDR
    self.registros["R1"] = self.MDR
    print(f"T6: R1 <- MDR | R1 = {self.registros['R1']}")

    print(f"\nEstado Final: PC=0x{self.PC:04X}, R1={self.registros['R1']} (Correcto)")

# Correr simulación
sim = SimuladorCicloInstruccion()
sim.trazar_ejecucion()

```

5. Ejercicios Resueltos

Ejercicio 1

Escribir las microoperaciones detalladas en lenguaje RTL para la ejecución completa de una instrucción de escritura en memoria: **SW R1, [2000]** (Store Word: guarda el contenido del registro **R1** en la dirección de memoria **2000**). Asumir que la fase Fetch (T_1 - T_3) ya ha concluido.

Solución: Una vez cargada la instrucción en el IR, las fases de la instrucción **SW** en los siguientes ciclos de reloj son:

- T_4 : $MAR \leftarrow \text{Dirección}(2000)$ (Se coloca la dirección destino en el registro MAR conectado al bus de direcciones).

- $T_5 : MDR \leftarrow R1$ (Se copia el contenido del registro de datos **R1** al MDR conectado al bus de datos).
- $T_6 : M[MAR] \leftarrow MDR$ (La Unidad de Control activa la señal de escritura **Write** en el bus de control, ordenando a la memoria RAM capturar el dato del bus de datos e introducirlo en la celda direccionada).

Ejercicio 2

Un procesador funciona a una frecuencia de reloj de 2 GHz (duración del ciclo de reloj $T = 0.5$ ns). Los retardos de conmutación física de sus componentes son: acceso a memoria = 0.4 ns, cálculo en ALU = 0.25 ns, acceso al banco de registros = 0.1 ns. Determinar cuál de estos componentes limita la frecuencia máxima del reloj si quisiéramos acelerar el procesador.

Solución: El componente que limita la frecuencia es aquel que tiene el mayor retardo de propagación, ya que define el camino crítico de la instrucción:

1. El retardo del componente más lento es el de **Acceso a Memoria** (0.4 ns).
2. Para que una microoperación de acceso a memoria se complete de forma estable, el ciclo de reloj T debe ser mayor que el retardo de dicho componente:

$$T \geq 0.4 \text{ ns}$$

3. La frecuencia máxima teórica permitida es:

$$f_{\text{máx}} = \frac{1}{0.4 \times 10^{-9} \text{ s}} = 2.5 \times 10^9 \text{ Hz} = 2.5 \text{ GHz}$$

Por lo tanto, la velocidad de reloj no puede superar los 2.5 GHz a menos que se optimice el tiempo de acceso a la memoria (por ejemplo, introduciendo memorias caché SRAM más rápidas).

6. Ejercicios Propuestos

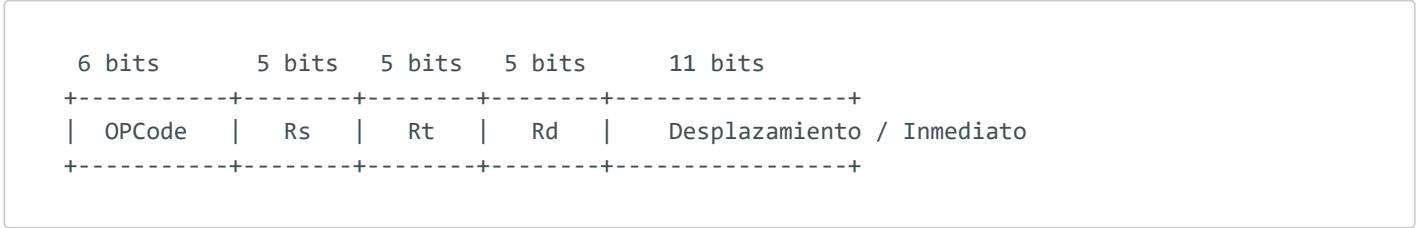
1. Escribe las microoperaciones en lenguaje RTL para el ciclo Fetch de una CPU cuyo tamaño de instrucción en memoria es de 2 bytes (16 bits) en lugar de 4 bytes.
2. Deduce el flujo en RTL para una instrucción de bifurcación incondicional **JUMP 0x1500** (salto de instrucción, que modifica directamente el flujo de ejecución copiando el valor de destino al PC).
3. Explica cómo implementa la Unidad de Control la fase de Decodificación (Decode) utilizando un decodificador binario conectado a los bits superiores del Registro de Instrucción (IR).

Tema 13: Introducción a la Programación en Lenguaje Ensamblador

El lenguaje ensamblador es la representación más baja y cercana al hardware de la programación de software. Cada instrucción en ensamblador se corresponde de forma directa (mapeo uno a uno) con una instrucción binaria ejecutable por la CPU (código de máquina). Aprender a programar en ensamblador permite entender cómo se manipulan físicamente los registros, cómo se gestiona la memoria RAM y cómo se implementan a nivel de hardware las estructuras lógicas de control (bucles e `if-else`) de los lenguajes de alto nivel.

1. Anatomía de una Instrucción de Máquina

Una instrucción de máquina es una palabra binaria dividida en campos específicos:



- **OPCODE (Código de Operación):** Bits que indican a la Unidad de Control la operación a realizar (ej. Suma, Carga, Salto).
- **Campos de Operandos:** Identifican qué registros del procesador o qué direcciones de memoria contienen los datos sobre los que se opera.
 - `Rs` (Source), `Rt` (Target): Registros fuente de operandos.
 - `Rd` (Destination): Registro de destino donde se guardará el resultado.

2. Modos de Direccionamiento

Definen la forma en que la instrucción calcula la dirección física del dato (operando) al que desea acceder:

1. **Direccionamiento Inmediato:** El propio dato está dentro de la instrucción. Es extremadamente rápido al no requerir accesos a registros o memoria.
 - *Ejemplo:* `ADD R1, R2, #5` (Suma el valor constante 5 a `R2` y guarda en `R1`).
2. **Direccionamiento Directo a Registro:** El dato reside en un registro del procesador.
 - *Ejemplo:* `ADD R1, R2, R3` (Suma el contenido de `R2` y `R3` y guarda en `R1`).

3. **Direccionamiento Directo (o Absoluto):** La instrucción contiene la dirección exacta de memoria RAM del dato.
- *Ejemplo:* `LD R1, [0x2000]` (Carga en `R1` el dato de la celda de memoria `0x2000`).
4. **Direccionamiento Indirecto a Registro (o Indexado):** La dirección de memoria del dato se calcula sumando una constante de desplazamiento al contenido de un registro base. Es el modo utilizado para recorrer vectores (arrays) y estructuras en memoria.
- *Ejemplo:* `LW R1, 4(R2)` (La dirección de lectura es $R2 + 4$).
-

3. Repertorio de Instrucciones Básico (Estilo MIPS / RISC-V)

Para construir algoritmos, empleamos las instrucciones básicas del procesador:

- **Carga y Almacenamiento:** `LW` (Load Word - lee de memoria a registro), `SW` (Store Word - escribe de registro a memoria).
 - **Aritmético-Lógicas:** `ADD` (Suma), `SUB` (Resta), `AND`, `OR`.
 - **Bifurcaciones (Saltos Condicionales):**
 - `BEQ R1, R2, Etiqueta` (Branch if Equal: salta a `Etiqueta` si $R1 = R2$).
 - `BNE R1, R2, Etiqueta` (Branch if Not Equal: salta a `Etiqueta` si $R1 \neq R2$).
 - **Saltos Incondicionales:** `J Etiqueta` (Jump: salta a `Etiqueta` sin comprobar condiciones).
-

4. El Toque Informático

El Ciclo de Traducción de Software: Compilador, Ensamblador y Enlazador

Cuando escribes código en un lenguaje de alto nivel como C++, el programa no se ejecuta de forma directa en el procesador. Pasa por las siguientes fases de traducción:

```
[Código C++] -> (Compilador) -> [Código Ensamblador] -> (Ensamblador) -> [Código Objeto (.o)]  
-> (Enlazador) -> [Ejecutable (.exe)]
```

1. **Compilador:** Traduce el código de alto nivel a un archivo de texto en lenguaje ensamblador específico de la arquitectura (ej. x86 o ARM).
2. **Ensamblador (Assembler):** Traduce los nemotécnicos de ensamblador a código binario binario de máquina ejecutable por el procesador, generando un archivo objeto (`.o` o `.obj`).

3. **Enlazador (Linker):** Junta varios archivos objeto y resuelve las llamadas a funciones externas de librerías del sistema, unificándolos en el archivo ejecutable final (**.exe** en Windows o ELF en Linux).

A continuación, presentamos un programa completo escrito en Ensamblador MIPS que implementa un bucle iterativo para calcular la suma de los primeros 10 números enteros positivos (equivalente a **for (int i=1; i<=10; i++) sum += i;**).

```
# PROGRAMA EN ENSAMBLADOR MIPS: Suma de los primeros 10 enteros
# Registros utilizados:
#  $t0: Contador del bucle (i)
#  $t1: Acumulador de la suma (sum)
#  $t2: Límite del bucle (10)

.text
.globl main

main:
    # 1. Inicialización
    li $t0, 1          # Carga inmediata: contador i = 1
    li $t1, 0          # Carga inmediata: sum = 0
    li $t2, 10         # Carga inmediata: límite = 10

bucle:
    # 2. Condición de parada del bucle
    # Si i > 10 (es decir, si no se cumple i <= 10), salimos del bucle
    # Para ello, si i es mayor que 10, salimos.
    # En MIPS simplificado: si i ($t0) es igual a 11 ($t2 + 1), salimos
    # O más sencillo: si i es mayor que 10, salimos.
    # Usamos BEQ para comprobar si el contador superó al límite

    # Suma: sum = sum + i
    add $t1, $t1, $t0  # t1 = t1 + t0

    # Incremento del contador: i = i + 1
    addi $t0, $t0, 1   # t0 = t0 + 1 (direccionamiento inmediato)

    # ¿Hemos terminado? Comprobamos si i <= 10
    # Si t0 <= t2, volvemos a la etiqueta 'bucle'
    # MIPS no tiene salto menor directo, evaluamos la condición contraria:
    # Si t0 (i) es menor o igual a t2 (10), saltamos de nuevo a bucle
    # Usamos un truco lógico o instrucción de comparación:
    ble $t0, $t2, bucle # Branch if Less or Equal: si t0 <= t2, salta a bucle

fin:
    # El resultado final queda almacenado en el registro $t1
    # Terminar ejecución del programa (llamada al sistema en MIPS)
    li $v0, 10
    syscall
```

5. Ejercicios Resueltos

Ejercicio 1

Traducir la siguiente estructura condicional de C++ a lenguaje Ensamblador MIPS simplificado:

```
if (a == b) {  
    c = a + 5;  
} else {  
    c = b - 2;  
}
```

Asumir que las variables *a*, *b* y *c* están almacenadas en los registros *\$t0*, *\$t1* y *\$t2* respectivamente.

Solución: Traducimos utilizando instrucciones de salto condicional e incondicional:

```
# Comparamos a ($t0) y b ($t1).  
# Si NO son iguales (Branch if Not Equal), saltamos a la rama del 'else'  
bne $t0, $t1, rama_else  
  
rama_if:  
# Si eran iguales, se ejecuta esto: c = a + 5  
addi $t2, $t0, 5    # t2 = t0 + 5  
j fin_condicional   # Salto incondicional para evitar ejecutar el 'else'  
  
rama_else:  
# Si no eran iguales, se ejecuta esto: c = b - 2  
addi $t2, $t1, -2    # t2 = t1 - 2 (la resta se hace sumando un número negativo)  
  
fin_condicional:  
# Continuación del programa...
```

Ejercicio 2

Identificar el modo de direccionamiento utilizado en cada una de las siguientes instrucciones del procesador:

1. *LW R1, [0x1024]*
2. *ADD R1, R2, #100*
3. *LW R1, 8(R3)*

Solución:

1. *LW R1, [0x1024]*: **Direccionamiento Directo (o Absoluto)**. La instrucción contiene directamente la dirección física de memoria (*0x1024*) a la que se desea acceder.
2. *ADD R1, R2, #100*: **Direccionamiento Inmediato** (para el operando *#100*). El valor constante 100 está contenido directamente dentro de la palabra de la instrucción. Para los registros *R1* y *R2* se utiliza **Direccionamiento Directo a Registro**.
3. *LW R1, 8(R3)*: **Direccionamiento Indirecto a Registro (o Indexado)**. La dirección efectiva del dato en memoria se calcula sumando el desplazamiento constante 8 al contenido almacenado en el registro base *R3*.

6. Ejercicios Propuestos

1. Escribe el código en lenguaje ensamblador MIPS correspondiente a un bucle **while** que reste de 5 en 5 un número guardado en el registro **\$t0** hasta que su valor sea menor o igual a cero.
2. Traduce a código ensamblador MIPS la siguiente operación aritmética: $c = (a + b) - (d + 8)$ asumiendo correspondencia a registros **\$t0** a **\$t3**.
3. Explica la diferencia entre un **Repertorio de Instrucciones CISC (Complex Instruction Set Computer)** y uno **RISC (Reduced Instruction Set Computer)** en términos de la duración de ciclo de instrucción y del formato de codificación en binario de las instrucciones.

Glosario de Términos

- **Algoritmo de Dijkstra:** (Nota: Término incluido por consistencia en la unificación general). Algoritmo codicioso para calcular caminos mínimos.
- **Arquitectura de Von Neumann:** Modelo clásico de computadores donde la CPU, la memoria (datos e instrucciones comparten el mismo bus físico) y el subsistema de Entrada/Salida están unificados.
- **Ciclo de Instrucción:** Proceso secuencial iterativo (Fetch, Decode, Execute, Write-Back) mediante el cual la CPU busca una instrucción en memoria, la decodifica y la ejecuta.
- **Complemento a 2:** Sistema de codificación binaria para enteros con signo donde los números positivos se codifican en binario puro y los negativos invirtiendo sus bits y sumando 1. Elimina el doble cero y simplifica las restas.
- **Contador de Programa (PC):** Registro de control de la CPU que almacena la dirección de memoria de la siguiente instrucción a procesar.
- **Cuello de Botella de Von Neumann:** Limitación física de velocidad de transferencia de datos causada porque las instrucciones y los datos comparten el mismo bus físico de memoria principal.
- **Decodificador:** Bloque combinacional con n entradas y 2^n salidas que activa únicamente la salida correspondiente al valor binario del bus de entrada.
- **Estándar IEEE 754:** Especificación técnica de la industria para representar números reales en coma flotante mediante campos de signo, exponente en exceso y mantisa normalizada con bit oculto.
- **Flip-Flop:** Celda básica de memoria síncrona sensible únicamente a los flancos de subida o bajada de una señal de reloj (CLK).
- **Lenguaje de Transferencia entre Registros (RTL):** Notación matemática utilizada para describir el flujo detallado de microoperaciones entre los registros del procesador.
- **Lenguaje Ensamblador:** Lenguaje de programación de bajo nivel que traduce los códigos de máquina binarios directos en nemotécnicos legibles por humanos.
- **Mapa de Karnaugh:** Herramienta gráfica bidimensional estructurada en código Gray utilizada para simplificar algebraicamente funciones lógicas.
- **Máquina de Estados Finitos (FSM):** Modelo matemático secuencial compuesto por estados estables, lógica de transición y lógica de salida.
- **Multiplexor (MUX):** Bloque lógico combinacional selector de datos con 2^n entradas de datos, n líneas de selección y una única salida.
- **UART:** Chip de interfaz de comunicaciones (transmisor-receptor asíncrono universal) que convierte datos serie de un solo hilo a buses paralelos anchos de la CPU y viceversa.

Bibliografía Recomendada

1. **Harris, D. M., & Harris, S. L. (2012).** *Digital Design and Computer Architecture* (2nd ed.). Morgan Kaufmann.
 - *Nota:* Un texto magistral y de gran claridad pedagógica que vincula de forma excelente el diseño digital de compuertas con la arquitectura del procesador.
2. **Patterson, D. A., & Hennessy, J. L. (2013).** *Computer Organization and Design: The Hardware/Software Interface* (5th ed.). Morgan Kaufmann.
 - *Nota:* La biblia académica fundamental e internacional de organización de computadores y programación en ensamblador MIPS/RISC-V.
3. **Stallings, W. (2015).** *Organización y arquitectura de computadores* (10ª ed.). Pearson.
 - *Nota:* Excelente manual muy riguroso para profundizar en el ciclo de instrucción, buses del sistema y la Unidad de Control.
4. **Floyd, T. L. (2006).** *Fundamentos de sistemas digitales* (9ª ed.). Prentice Hall.
 - *Nota:* Un libro de cabecera imprescindible para dominar compuertas, mapas de Karnaugh, bloques combinacionales MSI y biestables.
5. **Mano, M. M., & Ciletti, M. D. (2013).** *Digital Design* (5th ed.). Pearson.
 - *Nota:* Obra de gran prestigio para asimilar el diseño lógico de sistemas combinacionales y secuenciales síncronos.